



OPEN Grey wolf optimizer with self-repulsion strategy for feature selection

Yufeng Wang^{1,2}, Yumeng Yin²✉, Hang Zhao², Jinxuan Liu², Chunyu Xu^{3,4} & Wenyong Dong⁵

Feature selection is one of the most critical steps in big data analysis. Accurately extracting correct features from massive data can effectively improve the accuracy of big data processing algorithms. However, traditional grey wolf optimizer (GWO) algorithms often suffer from slow convergence and a tendency to fall into local optima, limiting their effectiveness in high-dimensional feature selection tasks. To address these limitations, we propose a novel feature selection algorithm called grey wolf optimizer with self-repulsion strategy (GWO-SRS). In GWO-SRS, the hierarchical structure of the wolf pack is flattened to enable rapid transmission of commands from the alpha wolf to each member, thereby accelerating convergence. Additionally, two distinct learning strategies are employed: the self-repulsion learning strategy for the alpha wolf and the pack learning strategy based on the predatory behavior of the alpha wolf, facilitating rapid self-learning for both the alpha wolf and the pack. These improvements effectively mitigate the weaknesses of traditional GWO, such as premature convergence and limited exploration capability. Finally, we conduct a comparative experimental analysis on the UCI test dataset using five relevant feature selection algorithms. The results demonstrate that the average classification error of GWO-SRS is reduced by approximately 15% compared to related algorithms, while utilizing 20% fewer features. This work highlights the need to address the inherent limitations of GWO and provides a robust solution to complex feature selection problems.

Keywords Grey wolf optimizer, Feature selection, Self-repulsion strategy, Transfer function

With the rapid development of computer technology and information life, a large number of high-dimensional data are generated. How to handle this data has become a complex and difficult problem to solve. These data contains a lot of irrelevant or redundant information, so it is particularly important to reduce the data scale on the premise of ensuring accurate data performance¹. To address the above issues, feature selection has been applied in many research fields, such as text analysis², image retrieval³, intrusion detection⁴, gene expression⁵, etc.

Feature selection is a process of selecting the most effective features from the original dataset to reduce the dimensionality of the data sets. Feature selection methods are usually split into three categories: filter method⁶, wrapper method⁷ and embedded method⁸. Filter method is based on the correlation or statistics between feature selection and target variables to select, it is to evaluate and rank each feature, and then select a subset according to a fixed number or threshold⁹. Wrapper method is to evaluate the performance by repeatedly trying different feature subsets during the training process, and then select the best feature subset according to the performance index¹⁰. Embedded method integrates the feature selection process with the training process of the learning algorithm, constrains the complexity of the model through regularization or other ways, and automatically selects the features with better predictive ability for the target variables¹¹.

In general, feature selection is considered as a search optimization problem, and the search space for a feature set of size n is 2^n . In order to deal with this situation, various methods such as exhaustive search, greedy search and random search have been proposed, but most of them have problems such as high complexity and large amount of calculation¹². In recent years, meta-heuristic algorithms have attracted a lot of attention due to their simplicity and flexibility¹³. Many research-ers have found that the combination of meta-heuristic algorithm and feature selection wrapper method has high research value. The Genetic Algorithm (GA), for instance, can

¹Academy for Electronic Information Discipline Studies, Nanyang Institute of Technology, Changjiang Road, Nanyang 473000, Henan, China. ²School of Software and Computer Science, Nanyang Institute of Technology, Changjiang Road, Nanyang 473000, Henan, China. ³Electronic Information School, Wuhan University, Bayi Road, Wuhan 430072, Hubei, China. ⁴School of Information Engineering, Nanyang Institute of Technology, Changjiang Road, Nanyang 473000, Henan, China. ⁵Computer Science School, Wuhan University, Bayi Road, Wuhan 430072, Hubei, China. ✉email: 2115925263@nyist.edu.cn

assess the quality of feature subsets by representing them as chromosomes and defining fitness functions¹⁴. It can search for the optimal feature subsets through crossing and mutation operations¹⁵. Barhoush et al.¹⁶ proposed an improved discrete salp swarm algorithm that enhances performance in feature selection for intrusion detection systems by introducing exploration and exploitation techniques. Faris et al.¹⁷ introduced an efficient binary salp swarm algorithm with a crossover scheme to address feature selection problems. In particle swarm optimization algorithm (PSO), each particle represents a feature subset which is updated based on its historical individual optimal location and the optimal location of the entire population to gradually optimize the quality of the feature subset¹⁸. In ant colony optimization (ACO), each ant represents a subset of features, selects the features based on pheromones and heuristic information, and guides the choices of other ants by updating the pheromones¹⁹. In simulated annealing Algorithm (SA), the algorithm can randomly jump out during the search process to explore the feature subset space²⁰.

Grey wolf optimizer (GWO) is a swarm intelligence optimization algorithm. It is characterized by a simple structure, a small number of parameters to set, and strong optimization ability²¹. Research shows that its optimization ability is significantly superior to traditional algorithms such as particle swarm optimization (PSO), Gravitational Search Algorithm (GSA) and differential evolution (DE)²². In recent years, many researchers have made many improvements to the GWO. Abdel-Basset et al.²³ presented an improved binary grey-wolf optimizer integrated with simulated annealing for feature selection, aiming to enhance the algorithm's global search ability and prevent premature convergence. Similarly, Al-Wajih et al.²⁴ proposed a hybrid binary grey wolf with harris hawks optimizer, combining the social hierarchy of GWO with the persistence of harris hawks optimizer, to address the challenges in feature selection. Al-Tashi et al.²⁵ further explored the potential of hybrid GWO by developing a binary optimization framework using hybrid grey wolf optimization for feature selection, showcasing its effectiveness in high-dimensional datasets. Kazem et al.²⁶ introduced an adaptive grey wolf optimizer, which adjusts the algorithm's parameters dynamically to better suit the problem at hand, thus improving the convergence speed and solution quality. Abdel-Basset et al.²⁷ also contributed to the field by fusing the grey wolf optimizer with a two-phase mutation strategy for feature selection, resulting in a more robust and efficient algorithm. Al-Wajih et al.²⁸ applied the binary grey wolf optimizer in conjunction with the K-nearest Neighbor classifier for feature selection, highlighting its utility in real-world applications. Too and Abdullah²⁹ utilized an opposition based competitive grey wolf optimizer for EMG feature selection, demonstrating the algorithm's ability to handle biomedical signal processing tasks. Latha et al.³⁰ proposed a hybrid binary gray wolf optimization approach for finding optimal features in classification problems, emphasizing the algorithm's versatility across different domains. Narinder Singh and S. B. Singh³¹ worked on a hybrid algorithm that combines particle swarm optimization with the grey wolf optimizer to improve convergence performance. Lastly, Abasi et al.³² focused on improving text feature selection for clustering using a binary grey wolf optimizer, underlining the algorithm's applicability in text mining and clustering tasks. Hu et al.³³ propose an improved binary grey wolf optimizer algorithm to solve the feature selection problem. Wang et al.³⁴ proposed an improved BGWO incorporating a novel population adaptation strategy and designed three strategies. Tripathi et al.³⁵ proposed a binary grey wolf optimization algorithm that integrates opposition strategies and weighted positioning to further improve the efficiency and accuracy of feature selection. At present, the existing BGWO algorithms are not comprehensive enough in addressing the relationship between early exploration and later exploitation, and the number of wolves used to adjust redundant features based on the characteristics of feature selection is also insufficient.

This paper focuses on how to balance the relationship between exploration and exploitation in binary grey wolf optimizer algorithm, and makes efforts to avoid local optimization. In addition, each feature of the elite head wolf is analyzed to improve the optimal solution. The main contributions of this paper are summarized as follows:

- A new wolf pack hierarchy is created. The layer of the wolf pack has been flattened from the original four layers to the current three layers. Through a learning strategy centered around the dominant head wolves, commands from the head wolves can quickly transmit to each wolf in the pack, resulting in faster convergence.
- A self-repulsion learning strategy based on an elite head wolf is proposed. This strategy considers the degree of influence of individual characteristics of the head wolf on its behavioral decisions. It implements an effective feature selection mechanism, eliminating the least relevant or redundant features. This strategy can reduce the error rate in the classification process and minimize the number of features used.
- A time-dependent hybrid transfer function is proposed. Initially, there is a higher probability of selecting 0, indicating the preference for selecting fewer features. As the process progresses, the probability of choosing 1 increases to ensure essential features are noticed. This approach effectively addresses the limitations of using a single transformation function.
- A novel nonlinear equation, combined with trigonometric functions, is introduced to calculate the convergence factor. This new approach can help GWO-SRS balance the gap between exploration and exploitation throughout the search process.
- A learning strategy based on head wolf plunder is proposed. In the wolf pack, the head wolf, as the leader, has a unique position and ability. Therefore, in the individual update stage, this study focuses on considering the leading role of this head wolf.

The remaining part is organized as follows. Section “[Related works](#)” introduces the relevant research of the algorithm used in the experiment. Section “[Background](#)” introduces the standard grey wolf optimizer algorithm and the binary grey wolf optimizer algorithm. Section “[The proposed GWO-SRS](#)” describes the five improvement strategies. The setup, results, and discussion of the experiment are given in Section “[Experimental results and analysis](#)”. Section “[Conclusion](#)” summarizes our work and proposes some suggestions for future work.

Related works

In recent years, various metaheuristic algorithms have been developed and applied to feature selection problems, demonstrating their effectiveness in improving search capabilities, accuracy, and stability. This section reviews the relevant studies on the algorithms used in our experiments, including Whale Optimization Algorithm (WOA), Ant Lion Optimizer (ALO), Sine Cosine Algorithm (SCA), and Brain Storm Optimization (BSO).

Yang et al.³⁶ proposed a multi-strategy assisted multi-objective whale optimization algorithm for feature selection. This approach enhances the search capability by combining multiple strategies, making it adaptable to the needs of multi-objective optimization problems. Additionally, Hussien et al.³⁷ introduced an S-shaped binary whale optimization algorithm, which improves the local search capability by incorporating an S-shaped transformation. This modification significantly enhances the accuracy of feature selection, particularly in high-dimensional datasets. Azar et al.³⁸ proposed a rough set-based ant lion optimizer that integrates rough set theory to enhance the performance of feature selection. This approach leverages the strengths of rough set theory in handling uncertainty and vagueness, making it suitable for complex feature selection tasks. Vashishtha and Kumar³⁹ further applied this algorithm to fault identification in centrifugal pumps, demonstrating its effectiveness in practical engineering problems. Their work highlights the versatility of ALO in addressing real-world challenges. Sun et al.⁴⁰ proposed a hybrid feature selection framework that combines an improved sine cosine algorithm with metaheuristic techniques. This framework enhances the efficiency and accuracy of feature selection by integrating various metaheuristic technologies, making it a robust solution for high-dimensional datasets. Kale and Uur⁴¹ investigated the update mechanisms of the sine cosine optimization algorithm and proposed advanced strategies to improve its feature selection capabilities in classification problems. Their work provides valuable insights into optimizing SCA for better performance in feature selection tasks. Li et al.⁴² proposed a stable feature selection method based on Brain Storm Optimization (BSO), which simulates the brainstorming process of human thought to improve the stability and search capability of the algorithm. This approach addresses the issue of instability in traditional feature selection methods, making it suitable for applications requiring consistent performance. Xue and Zhao⁴³ applied the brain storm optimization algorithm to feature selection in classification problems and studied the impact of structure and weight search on classification performance. Their work demonstrates the potential of BSO in improving classification accuracy through effective feature selection.

These studies collectively highlight the advancements in metaheuristic algorithms for feature selection, providing a solid foundation for our work.

Background

The standard grey wolf optimizer algorithm

The Grey Wolf Optimizer algorithm (GWO) is a meta-heuristic algorithm proposed by²². They found a strict social hierarchy in wolves by observing the behavior of the grey wolf pack in nature. GWO imitates the wolf pack's leadership hierarchy and prey mechanism, and the wolves are divided into four layers, denoted as alpha (α), beta (β), delta (δ) and omega (ω).

The wolf hierarchy is structured with the wolf (α) as the leader, responsible for decision-making such as hunting and resting. In contrast, the wolf (β) assists in decision-making and other collective activities. The wolf (ω) follows the decisions of the α and β wolves, and the remaining wolves, known as ω , obey orders. By mathematically modeling the grey wolf hierarchy, it is evident that the α -wolf represents the optimal solution, followed by the β -wolf as the second solution and the ω -wolf as the third solution. Each wolf updates its position based on the influence of the α , β and ω wolves. The position of a wolf is calculated as follows:

$$\vec{X}_i^{t+1} = \frac{\overrightarrow{XD1}_i^t + \overrightarrow{XD2}_i^t + \overrightarrow{XD3}_i^t}{3} \quad (1)$$

$$\overrightarrow{XD1}_i^t = \vec{X}_\alpha^t - A \cdot \vec{D}_\alpha^t \quad (2)$$

$$\overrightarrow{XD2}_i^t = \vec{X}_\beta^t - A \cdot \vec{D}_\beta^t \quad (3)$$

$$\overrightarrow{XD3}_i^t = \vec{X}_\omega^t - A \cdot \vec{D}_\omega^t \quad (4)$$

where $\vec{X}_i^{t+1}$ is the position vectors of i -th wolf at $(t+1)$ -th generation, $\vec{X}_\alpha^t$, $\vec{X}_\beta^t$ and $\vec{X}_\omega^t$ are the position vectors of α , β and ω wolves at t -th generation. A is the step size coefficient, it is calculated by Eq. (8). $\vec{D}_\alpha^t$, $\vec{D}_\beta^t$ and $\vec{D}_\omega^t$ is the distance between α , β , ω and i -th wolf at t -th generation, it is calculated as follows:

$$\vec{D}_\alpha^t = C \cdot |\vec{X}_\alpha^t - \vec{X}_i^t| \quad (5)$$

$$\vec{D}_\beta^t = C \cdot |\vec{X}_\beta^t - \vec{X}_i^t| \quad (6)$$

$$\vec{D}_\omega^t = C \cdot |\vec{X}_\omega^t - \vec{X}_i^t| \quad (7)$$

where C is a contraction coefficient, it is calculated as follows:

$$A = (2r_1 - 1) \cdot a \quad (8)$$

$$C = 2r_2 \quad (9)$$

$$a = 2 \cdot \left(1 - \frac{t}{MaxT}\right) \quad (10)$$

where r_1 and r_2 are random numbers between $[0,1]$, a is the convergence factor, which decreases linearly from 2 to 0 with the number of iterations, t is the current number of iterations, $MaxT$ is the maximum number of iterations.

Binary grey wolf optimizer

The search domain of the standard grey wolf optimizer algorithm is continuous. However, in feature selection problems, the value of its solution can only be 0 or 1. Therefore, when using the standard grey wolf optimizer algorithm to solve feature selection problems, it is necessary to encode and decode the solution, that is, the binary grey wolf optimizer (BGWO).

In the BGWO, the value interval of distance vectors ($\vec{D}_\alpha^t$, $\vec{D}_\beta^t$ and $\vec{D}_\omega^t$) is first compressed to a range between 0 and 1 by a transformation function. Then, the compressed distance vectors ($\vec{T}_\alpha^t$, $\vec{T}_\beta^t$ and $\vec{T}_\omega^t$) are mapped into binary distance vectors ($\vec{BD}_\alpha^t$, $\vec{BD}_\beta^t$ and $\vec{BD}_\omega^t$) by a selection operator, it is calculated as follows:

$$T_{\alpha,d}^t = \frac{1}{\left(1 + e^{-10(A \times D_{\alpha,d}^t - 0.5)}\right)} \quad (11)$$

$$T_{\beta,d}^t = \frac{1}{\left(1 + e^{-10(A \times D_{\beta,d}^t - 0.5)}\right)} \quad (12)$$

$$T_{\omega,d}^t = \frac{1}{\left(1 + e^{-10(A \times D_{\omega,d}^t - 0.5)}\right)} \quad (13)$$

where $T_{\alpha,d}^t$, $T_{\beta,d}^t$ and $T_{\omega,d}^t$ are the d -th dimension of $\vec{T}_\alpha^t$, $\vec{T}_\beta^t$ and $\vec{T}_\omega^t$ at t -th generation by using sigmoid function (called S_2), respectively. $D_{\alpha,d}^t$, $D_{\beta,d}^t$ and $D_{\omega,d}^t$ are the d -th dimension of $\vec{D}_\alpha^t$, $\vec{D}_\beta^t$ and $\vec{D}_\omega^t$ at t -th generation, respectively.

$$BD_{\alpha,d}^t = \begin{cases} 1, & \text{if } (T_{\alpha,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (14)$$

$$BD_{\beta,d}^t = \begin{cases} 1, & \text{if } (T_{\beta,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (15)$$

$$BD_{\omega,d}^t = \begin{cases} 1, & \text{if } (T_{\omega,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (16)$$

where $BD_{\alpha,d}^t$, $BD_{\beta,d}^t$ and $BD_{\omega,d}^t$ are the d -th dimension of $\vec{BD}_\alpha^t$, $\vec{BD}_\beta^t$ and $\vec{BD}_\omega^t$ at t -th generation, respectively. $Rand$ is random number between 0 and 1.

$$XD1_{\alpha,d}^t = \begin{cases} 1, & \text{if } (X_{\alpha,d}^t + BD_{\alpha,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (17)$$

$$XD2_{\beta,d}^t = \begin{cases} 1, & \text{if } (X_{\beta,d}^t + BD_{\beta,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (18)$$

$$XD3_{\omega,d}^t = \begin{cases} 1, & \text{if } (X_{\omega,d}^t + BD_{\omega,d}^t \geq Rand) \\ 0, & \text{else} \end{cases} \quad (19)$$

$$X_{i,d}^{t+1} = \begin{cases} XD1_{\alpha,d}^t, & \text{if } (Rand < \frac{1}{3}) \\ XD2_{\beta,d}^t, & \text{elseif } (\frac{1}{3} \leq Rand < \frac{2}{3}) \\ XD3_{\omega,d}^t, & \text{else} \end{cases} \quad (20)$$

where $X_{i,d}^{t+1}$ is the d -th dimension position of i -th wolf at $(t+1)$ -th generation, it calculated by a random cross selection method, as shown in Eq. (20).

The proposed GWO-SRS

New hierarchy of grey wolf pack

In nature, the grey wolf pack has a strict hierarchical system, and the division of layers determines the future direction of the entire wolf pack. In order to quickly convey the order of the alpha wolf to each wolf and improve the overall mobility of the wolf pack, we flattened and compressed the wolf pack hierarchy and proposed a new

wolf pack hierarchy. That is, the grey wolf pack is divided into three layers, namely alpha (α), beta (β), and omega (ω). The new hierarchy of grey wolf pack was shown in Fig. 1.

According to the fundamental survival law of survival of the fittest in the natural world, four sub-strong wolves are selected as candidate wolf (β) layers, named $\beta_1, \beta_2, \beta_3$ and β_4 respectively. Together, they form the second layer of the grey wolf pack. Among them, β_1 is the current optimal solution with the best fitness value, and it has the most chance of becoming the next generation's α wolf. β_2 is the sub-optimal solution, with the second-best fitness value. β_3 is the solution with the most significant decrease in fitness value compared to the previous generation. It is the fastest progressing solution. β_4 is the solution with the most significant change in fitness value. Then we will select the wolf with the best fitness value from the candidate wolf (β) layers, and consider it as the head wolf α , which dominates the wolf pack. ω are just ordinary wolves who obeys the orders of alpha wolf and beta wolves.

$$f(X_\alpha) = \min \{f(X_{\beta_1}), f(X_{\beta_2}), f(X_{\beta_3}), f(X_{\beta_4})\} \quad (21)$$

where X_α is the position of alpha wolf (α), $f(X_{\beta_1})$ is the fitness value of X_{β_1} .

$$f(X_{\beta_1}) = \min \{f(X_1), f(X_2), \dots, f(X_{NP})\} \quad (22)$$

where NP is the population size.

$$f(X_{\beta_2}) = \min \{f(X_1), f(X_2), \dots, f(X_{NP})\} - \{f(X_{\beta_1})\} \quad (23)$$

$$f(X_{\beta_3}) = \min_{1 \leq i \leq NP(X_i \neq X_{\beta_1}, X_i \neq X_{\beta_2})} f(X_i^t) - f(X_i^{t-1}) \quad (24)$$

$$f(X_{\beta_4}) = \max_{1 \leq i \leq NP(X_i \neq X_{\beta_1}, X_i \neq X_{\beta_2})} |f(X_i^t) - f(X_i^{t-1})| \quad (25)$$

where $f(X_i^t)$ is the fitness value of i -th wolf at t -th generation.

Self-repulsion learning strategy of elite wolves

In the flat hierarchy of the grey wolf pack, the elite wolves (head wolf α and candidate wolves β) lead the search direction of the wolf pack. The qualities of α and β are crucial to the regeneration of the offspring of the population. The suitable elite wolves can quickly guide other individuals in the population to converge to the global optimum.

The self-repulsion learning strategy of elite wolves selects the optimal mutation individuals by self-repulsing operation on its dimension. This strategy can help the elite wolves achieve fine-tuning and improve their performance. This approach focuses on eliminating redundant features in the data, which minimally contribute to the model's performance. By reducing data dimensionality and improving model efficiency, this strategy proves to be well-suited for feature selection. Feature selection typically involves a binary representation of feature selection status (1 is selection, 0 is no selection). This method can add to the theoretical framework of feature selection and provide a practical solution for processing high-dimensional data.

The main implementation process is to identify which features are selected by each elite wolf, and then determine the impact of the individual feature dimension on the overall classification error. After the elite wolf updates its position, it performs a feature inversion operation on each selected feature dimension of the individual. Then, the fitness value is calculated after each dimension is changed. Finally, the fitness value of the changed elite wolf is compared with that of the original elite wolf. If it is less than the fitness value of the original elite wolf, the changed elite wolf is retained. Otherwise, the original elite wolf is retained.

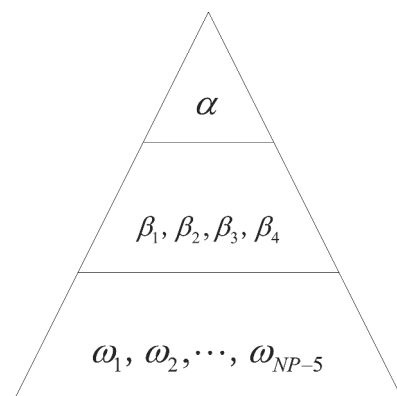


Fig. 1. Improved hierarchy of grey wolf pack.

```

1: Initialize parameters of GWO-SRS;
2: Randomly generate the positions of the wolves;
3: Compute the fitness value of each wolf;
4: Find  $X_{\beta 1}$ ,  $X_{\beta 2}$ ,  $X_{\beta 3}$  and  $X_{\beta 4}$  by Eqs. (22) – (25);
5: Select  $X_{\alpha}$  by Eq. (21);
6: for  $i = 1 : MaxT$  do
7:   Update  $A$ ,  $C$  and  $a$  by Eqs. (8) – (10);
8:   Calculate the new position of each wolf by Eqs. (11)–(19) (28)–(33);
9:   Calculate the fitness value of each wolf by Eq. (34);
10:  Update  $X_{\beta 1}$ ,  $X_{\beta 2}$ ,  $X_{\beta 3}$  and  $X_{\beta 4}$ ;
11:  for  $x$  in  $X_{\alpha}$ ,  $X_{\beta 1}$ ,  $X_{\beta 2}$ ,  $X_{\beta 3}$  and  $X_{\beta 4}$  do // Fine-tuning
12:     $m = x$ ;
13:    for  $d = 1:D$  do
14:      if  $m_d = 1$  then
15:        Change the value of  $m_d$  to 0;
16:        Calculate fitness value  $f(m)$ ;
17:      end if
18:      if  $f(m) < f(x)$  then
19:         $x = m$ ;
20:      end if
21:    end for
22:  end for
23:  Select  $X_{\alpha}$ ;
24: end for
25: Output  $X_{\alpha}$ 

```

Algorithm 1. GWO-SRS

For example, suppose the value of a wolf (a) is shown as in Fig. 2, its three feature dimensions are selected, and its fitness value are denoted as f_a . In the self-repulsion learning strategy, the wolf (a) is fine-tuned and turned into three wolves (b, c, and d). Wolf (b) is to change 1 to 0 at the second dimension of the wolf (a), which means that the second feature dimension of a is not selected. The fitness value of Wolf (b) is f_b . Wolf (c) is to change 1 to 0 at the third dimension of the wolf (a), and wolf (d) is to change 1 to 0 at the sixth dimension of the wolf (a). After fine-tuning, the smallest value f_{min} is selected from f_b , f_c and f_d . Then, f_a and f_{min} are compared. If f_a is the smallest, the wolf (a) remains unchanged and saves it. If f_{min} is small, it means that the fitness value of wolf (a) is worse than the fine-tuned wolf that removes some selected feature, and the changed wolf replaced wolf (a). The details of the proposed strategy are given in Algorithm 1, and the specific numerical results and data analysis can be found in section “Parameter settings”.

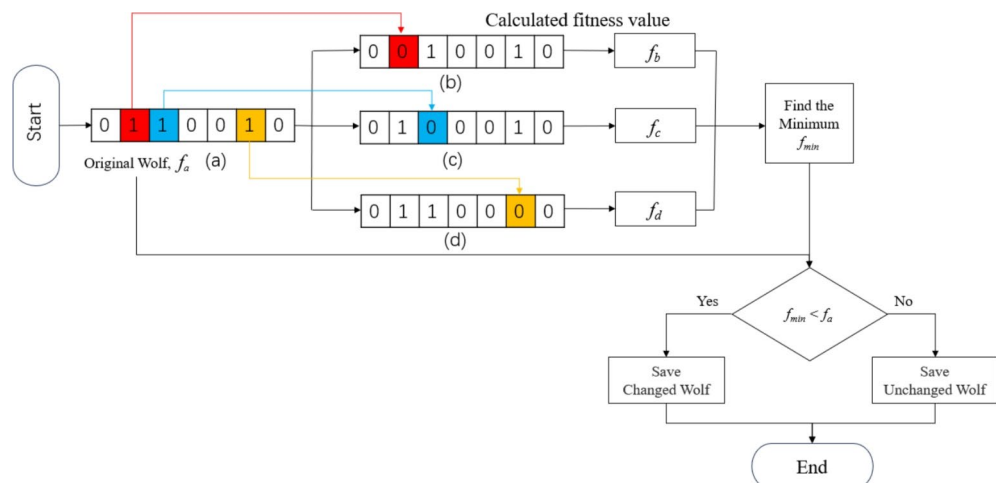


Fig. 2. Grey wolf self-repulsion flow chart.

Name	Improved transfer functions
$i_S_1(x)$	$\frac{1}{1+e^{-\frac{2x}{\varepsilon}}}$
$i_S_2(x)$	$\frac{1}{1+e^{-10(\frac{x}{\varepsilon}-0.5)}}$
$i_V_1(x)$	$\left \frac{\sqrt{17}\left(\frac{x}{\varepsilon}\right)}{4\sqrt{1+\left(\frac{x}{\varepsilon}\right)^2}} \right $
$i_V_2(x)$	$\frac{ \tanh(\frac{x}{\varepsilon}) }{\tanh(4)}$
$i_V_3(x)$	$\left \frac{\arctan(\frac{2x}{\pi\varepsilon})}{\arctan(2\pi)} \right $
$i_U(x)$	$\gamma \left \left(\frac{x}{\varepsilon} \right)^\mu \right (\gamma = 1, \mu = 0.5)$

Table 1. The details of improved transfer functions.

Name	Original transfer functions
$S_1(x)$	$\frac{1}{1+e^{-2x}}$
$S_2(x)$	$\frac{1}{1+e^{-10(x-0.5)}}$
$V_1(x)$	$\left \frac{\sqrt{17}x}{4\sqrt{1+x^2}} \right $
$V_2(x)$	$\frac{ \tanh(x) }{\tanh(4)}$
$V_3(x)$	$\left \frac{\arctan(\frac{2}{\pi}x)}{\arctan(2\pi)} \right $
$U(x)$	$\gamma (x)^\mu (\gamma = 1, \mu = 0.5)$

Table 2. The details of original transfer functions.

It can be seen from the above that the fitness value of elite wolves may change or remain unchanged after the self-repulsion learning strategy. It can ensure the self-learning ability of elite wolves. This strategy successfully reduces the number of features, reduces classification errors, and improves the leadership ability of elite wolves.

Improved transfer functions

The transfer function plays an important role in BGWO. According to Eq. (11), the independent variable of the transfer function is AD. To enhance comprehension, we rewrite AD as x . This section introduces three latest V shape functions, two S shape functions, and one U shape function. We first propose an improved time-dependent transfer function (see Table 1) to address the limitations of conventional approaches, then contrast it with existing static functions (Table 2).

In the above six transfer functions, the selection probability of the solution is constant throughout the evolution process. However, the task of the swarm intelligence algorithm is different in different stages at the whole iteration process. In the paper by Hu et al.³³, we learn that $x \in [-4, 4]$. In order to achieve a result for the transfer function that is as close to 0 in the initial stages and as close to 1 in the later stages. The contraction factor ε_t is a variable that decreases with iteration:

$$\varepsilon_t = 4 - \frac{4t}{MaxT} \quad (26)$$

where ε_t is a contraction factor at t -th generation, $MaxT$ is the maximum number of iterations.

Figure 3 demonstrates the time-varying curves of six improved transfer functions during 100 iterations with time step 5. The changes curve of four functions (V_1 , V_2 , V_3 , and U) contract from the outermost curve to the innermost curve gradually, and the degree of curvature of two functions (S_1 and S_2) increases gradually.

When x assumes a particular value, the early the transfer function provides a more extensive search space for the population, with a higher probability of 0 and a lower probability of 1. In the later stage, it has a high probability of being 1. The time-dependent transfer function ensures that the optimization process maintains a good balance between exploration and exploitation at each stage.

Nonlinear adaptive convergence factor

In the process of searching the optimal solution, exploration requires searching for more search space to increase population diversity and avoid falling into local optima. Exploitation improves the quality of solutions using promising solutions obtained through exploration to search for the best individuals around them. In other

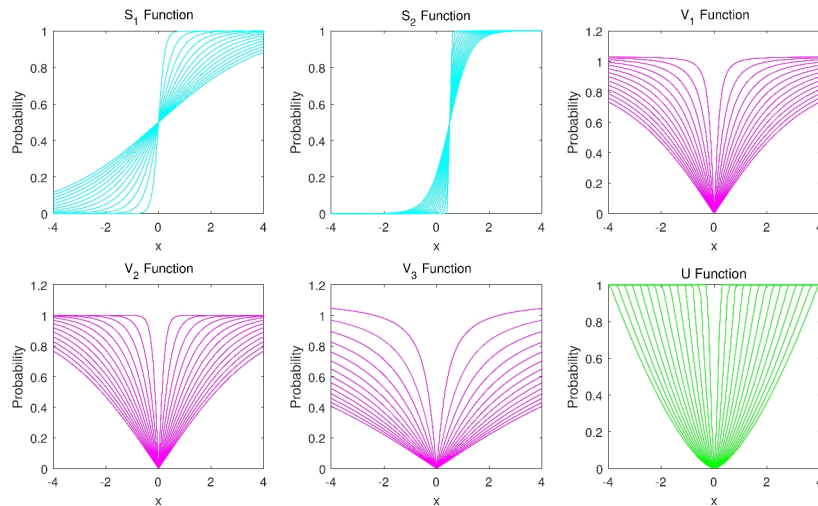


Fig. 3. The time-varying curves of improved transfer functions.

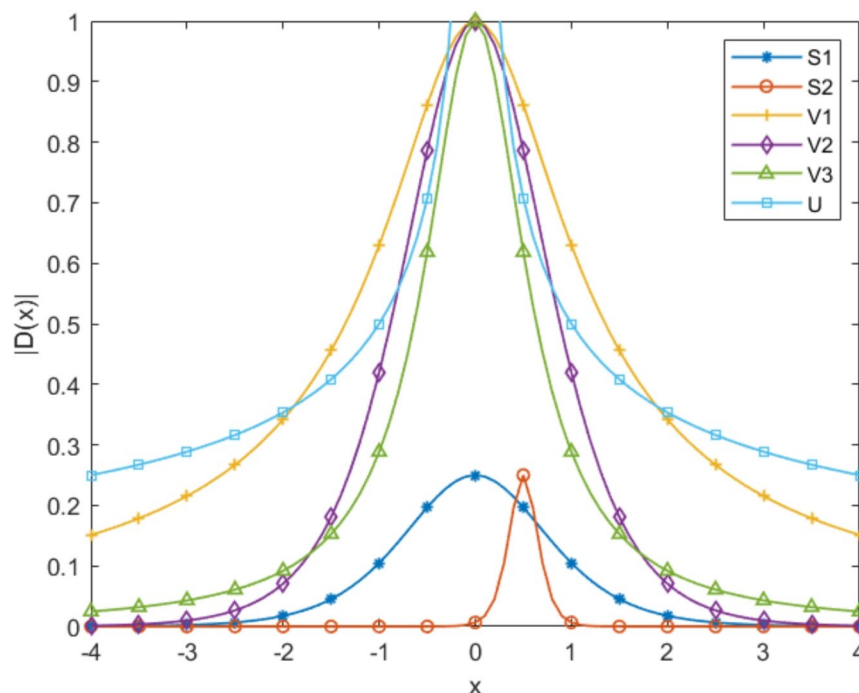


Fig. 4. The derivative curve of the improved transfer function.

words, in the early stage, the grey wolf should expand the search range by quickly switching the location. In the later stage of the algorithm, the speed of the grey wolf switching position becomes slower to find the optimal solution. GWO-SRS changes the position of the i -th wolf through the compressed distance $\bar{T}_i^t$ between α , β , ω and i -th wolf respectively. The compressed distance is calculated by the transfer function from the distance of two wolves, the curve shape of the transfer function represents the search preference.

In binary algorithms, the value of a position can only be selected from 0 or 1, and updating the position represents a transition between 0 and 1 in a discrete binary space. This transition is accomplished by altering the value of x in the transition function. The slope of the curve in Fig. 4 represents numerical acceleration. This figure indicates that the larger the absolute value of x , the smaller the value of $D(x)$, i.e., the smaller the slope; similarly, when the absolute value of x decreases, the slope increases. Since the slope represents the speed of position switching, if the absolute value of x is large, the wolf's position changes slowly; if the absolute value of x is small, the wolf's position changes rapidly.

Since feature selection is a highly complex problem, the linear convergence factor a can not better show the actual search process. In order to make the transfer function reflect different preference information at different

stages, a nonlinear adaptive convergence factor strategy is proposed. It can control the nonlinear adaptive change of parameter a , thus changing the curve shape of the transfer function at different search stages. This strategy can effectively balance the relationship between exploration and exploitation. The nonlinear update function of the convergence factor a is as follow:

$$a^t = \cos\left(\frac{t}{MaxT}\pi + \pi\right) + 1 \quad (27)$$

where a^t is the convergence factor at t -th generation, t is the number of current iterations and $MaxT$ is the maximum number of iterations. By using this equation, the value of convergence factor a can be adjusted nonlinearly during whole iteration. The curve of the convergence factor a is shown in Fig. 5.

Learning strategy based on head wolf plunder

Traditional binary grey wolf optimizer use the same probability of random crossover to determine the next generation position of an individual, but this does not reflect the importance of head wolf α and candidate wolf β in the population. In this section, we propose a learning strategy based on the predatory of the head wolf, so that head wolf α has an absolute power in the pack, the movement of α wolf determines the overall search direction of the pack. On the other hand, β wolf plays an auxiliary role to α wolf. The trajectory of β is more flexible and diverse compared to α . β wolf adjusts its position and movement according to α wolf's actions. It also interacts with other wolves like Omega wolves. The trajectory of β reflects its search behavior and strategic adjustments while assisting α wolf. It contributes to the overall search process by providing additional exploration and fine-tuning in different areas.

We use a roulette similar way to bring the position of the next generation of individuals closer to the head wolf. Since the fitness value sought is a minimization problem, the reciprocal form is used for each fitness value. Regarding a new individual update formula, it is as follows:

$$fit(x_i) = f_{max} - f(x_i) \quad (28)$$

where $fit(x_i)$ is the fitness D-value of the i -th wolf, f_{max} is the max fitness value of the whole wolf pack, $f(x_i)$ is the fitness value of the i -th wolf.

$$F_\alpha = fit(x_\alpha) \quad (29)$$

$$F_\beta = \sqrt{\frac{fit^2(x_{\beta 1}) + fit^2(x_{\beta 2}) + fit^2(x_{\beta 3}) + fit^2(x_{\beta 4})}{4}} \quad (30)$$

$$F_\omega = \sqrt{\frac{\sum_{j=1}^{NP-4} fit^2(x_j)}{NP - 4}} \quad (31)$$

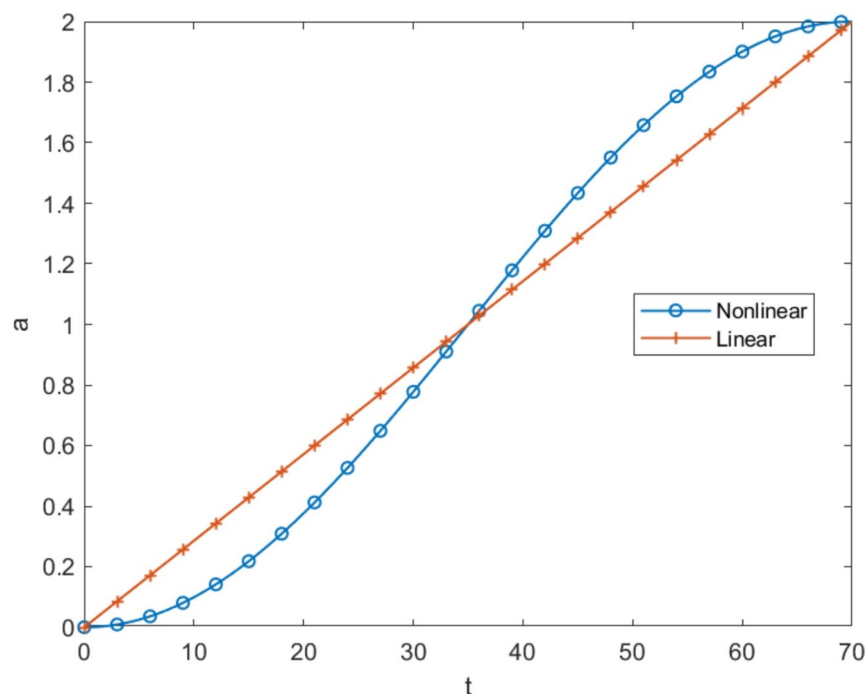


Fig. 5. The convergence factor curve of linear and nonlinear.

where

$$F = \frac{F_{\alpha} + F_{\beta} + F_{\omega}}{3} \tag{32}$$

$$X_{i,d}^{t+1} = \begin{cases} XD1_{\alpha,d}^t, & \text{if } (0 < Rand \leq \frac{F_{\alpha}}{F}) \\ R(XD2_{\beta,d}^t), & \text{elseif } (\frac{F_{\alpha}}{F} < Rand \leq \frac{F_{\alpha} + F_{\beta}}{F}) \\ R(XD3_{\omega,d}^t), & \text{else} \end{cases} \tag{33}$$

where $X_{i,d}^{t+1}$ is the d -th dimension position of i -th wolf at $(t + 1)$ -th generation, $R(XD2_{\beta,d}^t)$ represents the selection of the mapped value of a wolf's position from four β wolves ($XD2_{\beta1,d}^t$, $XD2_{\beta2,d}^t$, $XD2_{\beta3,d}^t$ and $XD2_{\beta4,d}^t$) randomly, $R(XD3_{\omega,d}^t)$ represents the selection of the mapped value of a wolf's position from $(NP - 4) \omega$ wolves randomly.

Computational complexity of GWO-SRS

The computational complexity of the GWO-SRS algorithm can be analyzed based on its key operations. The initialization phase involves generating the positions of the wolves and computing their fitness values, which has a complexity of $O(NP \times D)$, where NP is the number of wolves and D is the dimensionality of the problem. During each iteration, the algorithm updates the positions of the wolves using Eqs. (11)–(19) and (28)–(33), which involves operations on each dimension of each wolf, resulting in a complexity of $O(NP \times D)$. Additionally, the fitness value of each wolf is recalculated, contributing another $O(NP \times D)$ complexity. The selection and update of the alpha and beta wolves involve comparisons and updates, which add a complexity of $O(NP)$. The mutation step, where the value of m_d is changed and the fitness is recalculated, has a complexity of $O(NP \times D)$. Therefore, the overall computational complexity of GWO-SRS per iteration is $O(NP \times D)$. Given $MaxT$ iterations, the total complexity of the algorithm is $O(MaxT \times NP \times D)$. This makes GWO-SRS computationally efficient for problems with moderate dimensionality and population size.

Experimental results and analysis

The following section discusses the datasets, parameter settings, evaluation function, comparison with relevant algorithms, comparison of the performance of improved transfer functions, effectiveness of improved transfer functions, effectiveness of the nonlinear adaptive convergence factor and effectiveness of the learning strategy.

Datasets

The effectiveness and robustness of the algorithm we propose will be thoroughly investigated through feature selection using ten well-known datasets. These datasets originate from the UC Irvine Machine Learning Repository, which can be downloaded from the UCI datasets page (<http://archive.ics.uci.edu>). A brief description of the datasets used is provided in Table 3. For each dataset, details such as Instances, Features (number of features), Features types, Dataset Characteristics, and Missing values are included.

Parameter settings

Each algorithm is run 20 independent runs with a random seed. For all the subsequent experiments, the maximum number of iterations, denoted as $MaxT$, is set to 100. The population size, labeled as NP , is 7. The

Dataset	Instances	Features	Features types	Dataset characteristics	Missing values
Waveform Database Generator (Version 2)	5000	40	Real	Multivariate, Data-Generator	No
Breast Cancer Wisconsin (Diagnostic)	569	30	Real	Multivariate	No
Congressional Voting Records	435	16	Categorical	Multivariate	Yes
Ionosphere	351	34	Integer, Real	Multivariate	No
Lymphography	148	19	Categorical	Multivariate	No
Semeion Handwritten Digit	1592	265	Integer	Multivariate	No
SPECT Heart	267	22	Categorical	Multivariate	No
Tic-Tac-Toe Endgame	958	9	Categorical	Multivariate	No
Wine	178	13	Integer, Real	Tabular	No
Zoo	101	16	Categorical, Integer	Multivariate	No
Clean1	476	166	Integer	Multivariate	No
Clean2	6598	166	Integer	Multivariate	No
Exactly	1000	13	N/A	Multivariate	No
Exactly2	1000	13	N/A	Multivariate	No
Krvskp	3196	36	Categorical	Multivariate	Yes
Vote	300	16	N/A	Multivariate	No

Table 3. The details of the testing datasets.

dimension D of each test dataset corresponds to the number of features. Moreover, the number of neighbors in K-Nearest Neighbors (KNN) algorithm is 5, and 5-fold cross-validation is employed here.

Evaluation function

Feature selection aims to select the most representative, relevant, or effective feature subset from the original feature set to build models. It can improve model performance, reduce over-fitting, and speed up training. In other words, it is necessary to reduce both the number of features and the classification error. Therefore, in the evaluation function of feature selection, we choose this Eq. (34), and both the classification error and the number of features are considered.

$$Fitness = m * kflodLoss + \frac{n * |S|}{|C|} \quad (34)$$

where $kflodLoss$ is the classification error of cross validation, $|S|$ is the number of subsets in the feature, and $|C|$ is the number of datasets in the feature. m and n are two weight coefficients, where m is 0.99 and n is 0.01, followed by⁴⁴.

Comparison with relevant algorithms

In order to better validate the performance of the proposed method, GWO-SRS was compared with five relevant algorithms, including Binary Grey Wolf Optimizer (BGWO), Ant Lion Optimizer (ALO), Brain Storm Optimizer (BSO), Sine Cosine Algorithm (SCA), and Whale Optimization Algorithm (WOA). The data of these five algorithms comes from⁴⁵.

This section uses the Sigmoid (S_2) transfer function to compare GWO-SRS with the other five relevant algorithms. The experimental data are rounded to four decimal places for easy reading. Figure 6 shows the ranking distribution of classification errors of these six comparison algorithms, and Fig. 7 shows the ranking distribution of the number of features for these six comparison algorithms.

Table 4 shows the comparison results between GWO-SRS and the other five algorithms regarding classification error, where the column of Error represents the average classification error. The Rank column represents the ranking of the six comparison algorithms. The average classification errors of the six comparison algorithms are 1, 2, 3, 4, 5, and 6 in order from the best to the worst (the smallest value is the best). The Total row is the sum of the Rank obtained for each algorithm on ten test datasets. According to the results in Table 4, GWO-SRS demonstrates superior performance across most datasets, achieving the lowest classification errors and securing the top rank in 11 out of 16 datasets. With a total rank score of 24, GWO-SRS significantly outperforms other algorithms, highlighting its effectiveness in feature selection tasks. WOA follows as the second-best performer, with a total rank score of 32, and it particularly excels in datasets such as Krvskp and Wine. SCA and BALO show moderate performance, with total rank scores of 54 and 53, respectively, while BGWO and BSO exhibit relatively poorer performance, with total rank scores of 63 and 84. Notably, BSO struggles in datasets like Exactly and Exactly2, where its classification errors are notably higher. Overall, the results underscore the robustness and efficiency of GWO-SRS in addressing feature selection challenges, making it a highly effective approach compared to the other algorithms evaluated.

Table 5 shows the comparison results between GWO-SRS and the other five algorithms on the features' number, where the column of Number represents the average number of features. From Table 5, GWO-SRS consistently demonstrates superior performance in selecting fewer features across most datasets, achieving the lowest average number of features in 11 out of 16 datasets and securing a total rank score of 20. This highlights its efficiency in reducing feature dimensionality while maintaining performance. BSO also performs well, particularly in datasets like Wine, Zoo, Exactly, and Krvskp, where it achieves the lowest number of features,

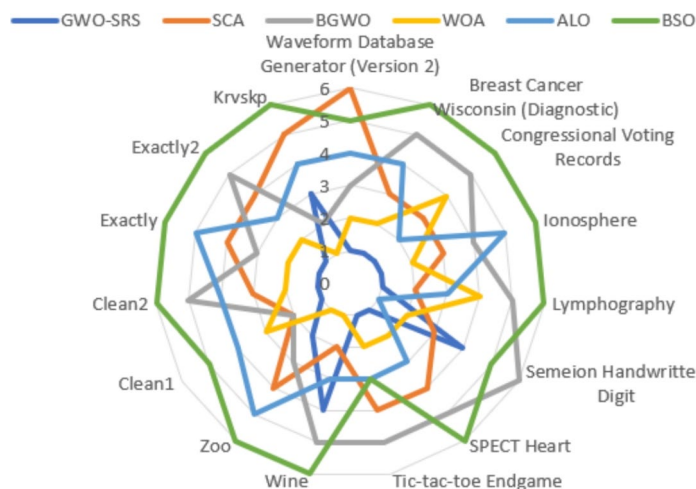


Fig. 6. Ranking distribution of the classification errors.

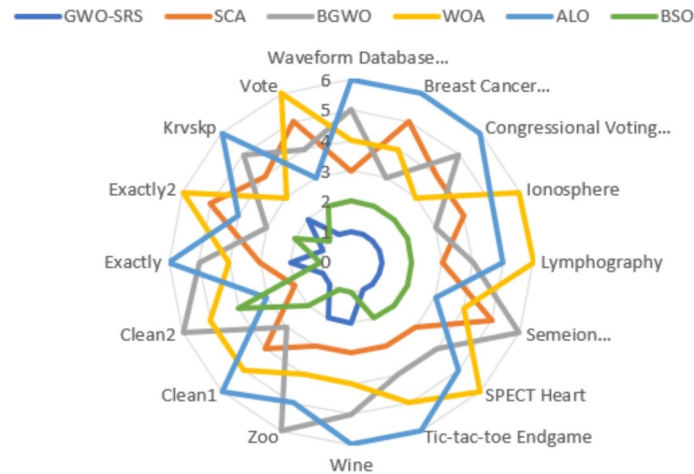


Fig. 7. Ranking distribution of the average number of features.

Dataset	GWO-SRS		SCA		BGWO		WOA		BALO		BSO	
	Error	Rank	Error	Rank	Error	Rank	Error	Rank	Error	Rank	Error	Rank
Waveform Database Generator (Version 2)	0.2436	1	0.3010	6	0.2973	3	0.2921	2	0.3000	4	0.3860	5
Breast Cancer Wisconsin (Diagnostic)	0.0284	1	0.0604	3	0.0680	5	0.0575	2	0.0608	4	0.0980	6
Congressional Voting Records	0.0276	1	0.0651	3	0.0730	5	0.0667	4	0.0630	2	0.1453	6
Ionosphere	0.0283	1	0.1174	3	0.1362	4	0.1152	2	0.1405	5	0.1462	6
Lymphography	0.1454	1	0.2120	2	0.2415	5	0.2342	4	0.2137	3	0.3069	6
Semeion Handwritten Digit	0.0385	4	0.0296	3	0.0400	6	0.0291	2	0.0286	1	0.0396	5
SPECT Heart	0.1617	1	0.2129	4	0.2191	5	0.2049	2	0.2119	3	0.2493	6
Tic-Tac-Toe Endgame	0.2325	1	0.2454	4	0.2526	5	0.239	2	0.3399	3	0.3399	3
Wine	0.0464	4	0.0434	2	0.0600	5	0.0345	1	0.0457	3	0.1348	6
Zoo	0.0480	2	0.0693	4	0.0554	3	0.0432	1	0.0784	5	0.1869	6
Clean1	0.1352	1	0.1472	2	0.1472	2	0.1564	3	0.1604	4	0.1771	5
Clean2	0.0480	1	0.0540	3	0.0601	5	0.0536	2	0.0553	4	0.0648	6
Exactly	0.2143	1	0.2853	4	0.2819	3	0.2621	2	0.3011	5	0.4000	6
Exactly2	0.2564	1	0.3061	4	0.3102	5	0.3030	2	0.3047	3	0.3672	6
Krvskp	0.0942	3	0.1107	5	0.0937	2	0.0860	1	0.1082	4	0.2421	6
Vote	0.0625	1	0.0848	4	0.0854	5	0.0823	3	0.0790	2	0.1624	6
Total		24		54		63		32		53		84

Table 4. Comparison between the proposed approaches based on classification errors. Significant values are in bold.

resulting in a total rank score of 30. WOA, SCA, and BALO show moderate performance, with total rank scores of 75, 59, and 81, respectively, while BGWO lags behind with a total rank score of 71. Notably, GWO-SRS excels in high-dimensional datasets such as Semeion Handwritten Digit and Clean1, where it significantly outperforms other algorithms. Overall, the results underscore the effectiveness of GWO-SRS in achieving efficient feature selection, making it a robust choice for dimensionality reduction tasks compared to the other algorithms evaluated.

Table 6 shows the comparison between the proposed GWO-SRS and relevant algorithms based on the average running time. Based on the data presented in the table, GWO-SRS demonstrates superior efficiency in terms of average running time across most datasets compared to the other algorithms. It achieves the lowest running time in datasets such as Waveform Database Generator (Version 2), Breast Cancer Wisconsin (Diagnostic), Congressional Voting Records, Ionosphere, Semeion Handwritten Digit, SPECT Heart, Tic-Tac-Toe Endgame, Wine, Zoo, Clean1, Exactly, Exactly2, Krvskp, and Vote. Notably, GWO-SRS significantly outperforms other algorithms in high-dimensional datasets like Clean2. While SCA, BGWO, WOA, BALO, and BSO show varying performance, they generally have longer running times, with BALO particularly struggling in Clean2. Overall, GWO-SRS proves to be the most computationally efficient algorithm, making it a robust choice for feature selection tasks, especially in high-dimensional and complex datasets.

In order to judge whether the experimental results are statistically significant, the independent *t* test method is used to make a comparison between GWO-SRS, SCA, BGWO, WOA, ALO, and BSO. Table 7 presents the

Dataset	GWO-SRS		SCA		BGWO		WOA		BALO		BSO	
	Number	Rank	Number	Rank	Number	Rank	Number	Rank	Number	Rank	Number	Rank
Waveform Database Generator (Version 2)	27.40	1	34.64	3	36.60	5	36.40	4	39.60	6	29.00	2
Breast Cancer Wisconsin (Diagnostic)	12.10	1	20.47	5	19.00	3	20.00	4	24.27	6	13.73	2
Congressional Voting Records	3.40	1	9.00	4	9.80	5	8.87	3	9.87	6	7.53	2
Ionosphere	10.05	1	19.07	4	17.63	3	21.67	6	20.13	5	15.93	2
Lymphography	7.15	1	10.87	3	11.80	4	14.20	6	13.33	5	9.47	2
Semeion Handwritten Digit	130.60	1	194.40	5	203.60	6	188.00	4	187.80	3	162.00	2
SPECT Heart	9.60	1	12.60	3	13.20	4	14.13	6	13.87	5	10.87	2
Tic-Tac-Toe Endgame	5.05	1	7.47	3	7.50	4	7.87	5	8.80	6	5.88	2
Wine	7.90	2	9.40	3	10.73	5	9.93	4	11.07	6	6.67	1
Zoo	7.75	2	9.60	3	12.40	6	10.93	4	11.67	5	7.67	1
Clean1	90.67	1	110.20	4	109.60	3	121.93	5	132.00	6	98.73	2
Clean2	91.73	1	93.40	2	106.00	6	102.00	5	95.00	3	101.00	4
Exactly	8.40	2	10.47	3	12.07	5	11.20	4	12.87	6	7.73	1
Exactly2	5.50	1	9.00	5	7.53	3	9.47	6	8.40	4	6.27	2
Krvskp	18.60	2	30.80	4	31.60	5	27.60	3	35.80	6	17.80	1
Vote	6.41	1	9.60	5	8.47	4	10.33	6	8.40	3	7.87	2
Total		20		59		71		75		81		30

Table 5. Comparison between the proposed approaches based on average number of features. Significant values are in bold.

Dataset	GWO-SRS	SCA	BGWO	WOA	BALO	BSO
Waveform Database Generator (Version 2)	15.34	43.72	20.63	86.64	40.51	25.03
Breast Cancer Wisconsin (Diagnostic)	1.56	2.41	3.61	2.35	2.87	2.85
Congressional Voting Records	1.48	2.59	3.32	2.59	2.88	3.33
Ionosphere	2.26	2.60	3.25	2.57	3.14	3.1
Lymphography	2.59	2.38	2.98	2.91	2.68	2.94
Semeion Handwritten Digit	16.52	24.06	31.67	19.21	28.41	14.33
SPECT Heart	2.19	2.38	3.00	2.38	2.88	2.96
Tic-Tac-Toe Endgame	3.71	4.38	4.38	4.1	4.36	3.99
Wine	2.48	2.43	3.13	2.47	2.68	2.92
Zoo	1.34	2.30	3.25	2.19	2.79	4.85
Clean1	2.58	3.61	3.39	3.54	5.31	3.58
Clean2	150.76	223.7	158.67	182.94	610.83	223.69
Exactly	2.64	4.63	4.04	4.65	3.92	4.58
Exactly2	3.75	4.88	4.52	4.22	4.22	4.62
Krvskp	7.86	15.89	9.53	13.03	18.16	11.56
Vote	2.50	2.60	3.25	2.47	2.89	3.26

Table 6. Comparison between the proposed GWO-SRS and relevant algorithms based on the average running time.

result of the t test with p values. Note that the GWO-SRS is used as the reference algorithm in this test. As can be seen, the classification performance of GWO-SRS was significantly better than SCA, BGWO, WOA, BALO, and BSO in most cases (p value < 0.05).

In general, the GWO-SRS algorithm is outstanding in this experiment. It has certain advantages in both error and number on multiple data sets. Other algorithms such as SCA, BGWO, WOA, ALO, and BSO also have their own characteristics and performances on different data sets, but overall, they are slightly inferior to the GWO-SRS algorithm. This experiment provides valuable references for the performance of different algorithms on different data sets.

Comparison of the performance of improved transfer functions

This experiment tests the performance of six improved transfer functions on GWO-SRS. Table 8 shows the comparison results of classification errors of different improved transfer functions. Table 9 compares the average number of features of different improved transfer functions. Figure 8 shows the ranking distribution of classification errors for different improved transfer functions. Figure 9 shows the ranking distribution of the

Dataset	SCA	BGWO	WOA	BSO	BALO
Waveform Database Generator (Version 2)	0.0854	0.1453	0.0632	0.0953	0.0563
Breast Cancer Wisconsin (Diagnostic)	0.1862	0.0946	0.0762	0.0849	0.0463
Congressional Voting Records	0.0764	0.0867	0.0941	0.0326	0.0756
Ionosphere	0.0876	0.0745	0.2183	0.0946	3.1851
Lymphography	0.6230	1.8946	1.6370	2.8964	1.7641
Semeion Handwritten Digit	0.9564	0.3421	0.7645	0.8942	0.5618
SPECT Heart	2.5697	3.4790	2.4836	1.9632	1.0654
Tic-Tac-Toe Endgame	0.9346	0.7643	0.0673	1.5624	1.1457
Wine	0.0478	0.0596	1.8934	0.9631	2.6972
Zoo	0.7963	0.9634	0.6792	0.7954	0.2586
Clean1	1.5624	3.8645	2.6478	2.5189	3.4751
Clean2	4.2571	5.1485	3.7456	3.8421	4.5876
Exactly	2.6984	4.6751	2.7641	3.6975	3.5427
Exactly2	5.6482	2.6479	4.6931	5.746	4.8159
Krvskp	3.6784	5.1627	5.6984	5.4163	4.7654
Vote	4.5195	5.7469	6.6873	4.3529	3.7684

Table 7. Experimental result of t test with p values.

Dataset	i_S_1		i_S_2		i_V_1		i_V_2		i_V_3		i_U	
	Error	Rank	Error	Rank	Error	Rank	Error	Rank	Error	Rank	Error	Rank
Waveform Database Generator (Version 2)	0.2452	2	0.2360	1	0.2564	6	0.2473	3	0.2494	5	0.2474	4
Breast Cancer Wisconsin (Diagnostic)	0.0289	6	0.0280	1	0.0285	4	0.0284	3	0.0286	5	0.0281	2
Congressional Voting Records	0.0416	6	0.0306	1	0.0345	3	0.0347	4	0.0348	5	0.0331	2
Ionosphere	0.1210	6	0.0994	3	0.1005	4	0.0957	2	0.1026	5	0.0935	1
Lymphography	0.1624	6	0.1453	1	0.1523	4	0.1477	2	0.1513	3	0.1579	5
Semeion Handwritten Digit	0.0732	6	0.0680	2	0.0673	1	0.0682	4	0.0684	5	0.0681	3
SPECT Heart	0.1494	1	0.1513	2	0.1673	5	0.1634	3	0.1713	6	0.1670	4
Tic-Tac-Toe Endgame	0.1845	1	0.2305	2	0.2312	3	0.2318	4	0.2553	6	0.2387	5
Wine	0.0494	3	0.0474	1	0.0560	6	0.0527	5	0.0503	4	0.0488	2
Zoo	0.0519	2	0.0554	3	0.0462	1	0.0635	5	0.0654	6	0.0615	4
Clean1	0.1296	3	0.1256	1	0.1335	4	0.1287	2	0.1367	5	0.1402	6
Clean2	0.0523	4	0.0482	2	0.0531	5	0.0497	3	0.0554	6	0.0473	1
Exactly	0.2251	2	0.2043	1	0.2337	3	0.2459	4	0.2473	5	0.2546	6
Exactly2	0.2590	5	0.2364	1	0.2447	4	0.2394	3	0.2594	6	0.2379	2
Krvskp	0.1094	4	0.1051	3	0.0764	1	0.1163	5	0.0865	2	0.1167	6
Vote	0.0827	4	0.0586	1	0.1072	5	0.1294	6	0.0749	3	0.0758	2
Total		61		26		59		58		77		55

Table 8. Comparison of the classification errors on different improved transfer functions. Significant values are in bold.

average number of features for different improved transfer functions. Figures 10, 11 and 12 shows the boxplots of the results obtained in different transfer functions for each dataset after 20 runs.

From Table 8, we can see that the improved transfer functions exhibit varying performance across different datasets. The transfer function i_S_2 demonstrates the best overall performance, achieving the lowest classification errors in 7 out of 16 datasets and securing a total rank score of 26. This highlights its effectiveness in enhancing classification accuracy. The transfer function i_V_3 also performs well, particularly in datasets like Krvskp, where it achieves the lowest classification error, contributing to its total rank score of 55. In contrast, the transfer function i_S_1 shows the poorest performance, with the highest classification errors in several datasets and a total rank score of 61. The transfer functions i_V_1 , i_V_2 , and i_U exhibit moderate performance, with total rank scores of 59, 58, and 77, respectively. Notably, i_V_1 excels in the Semeion Handwritten Digit and Zoo datasets, while i_U performs well in the Clean2 dataset. Overall, the results indicate that i_S_2 is the most effective transfer function for improving classification accuracy, while i_S_1 lags behind in performance.

By observing the average number of features in Table 9, we can see that the improved transfer functions exhibit varying performance in terms of the average number of features selected across different datasets. The transfer function i_S_2 demonstrates the best overall performance, achieving the lowest average number of

Dataset	i_S_1		i_S_2		i_V_1		i_V_2		i_V_3		i_U	
	Number	Rank	Number	Rank	Number	Rank	Number	Rank	Number	Rank	Number	Rank
Waveform Database Generator (Version 2)	32.15	5	27.40	2	26.90	1	31.25	4	34.00	6	30.10	3
Breast Cancer Wisconsin (Diagnostic)	13.95	5	12.70	4	11.35	1	12.40	2	11.35	1	12.50	3
Congressional Voting Records	7.30	5	3.85	4	3.10	1	3.25	2	3.35	3	3.85	4
Ionosphere	21.95	6	8.45	4	7.95	2	8.20	3	9.40	5	7.30	1
Lymphography	13.55	6	6.45	2	6.75	4	7.05	5	6.25	1	6.70	3
Semeion Handwritten Digit	151.80	6	128.75	4	128.15	3	127.02	1	130.60	5	127.75	2
SPECT Heart	18.60	6	9.75	4	8.65	2	8.30	1	9.85	5	9.40	3
Tic-Tac-Toe Endgame	9.00	5	5.20	3	4.90	2	5.40	4	4.90	2	4.75	1
Wine	9.65	6	4.20	2	4.45	3	4.55	4	4.60	5	4.10	1
Zoo	12.50	6	7.05	4	8.00	5	6.75	3	6.65	1	6.60	2
Clean1	98.76	5	92.64	2	96.43	4	92.81	3	91.53	1	101.61	6
Clean2	95.73	4	90.35	1	91.97	3	91.75	2	99.11	6	96.00	5
Exactly	9.51	5	7.71	2	8.93	4	6.54	1	8.30	3	9.73	6
Exactly2	8.56	4	5.41	1	8.97	5	9.10	6	6.23	2	6.57	3
Krvskp	20.50	6	18.69	3	18.54	2	19.87	4	20.01	5	15.76	1
Vote	7.36	4	6.50	1	6.93	3	6.64	2	8.12	5	8.57	6
Total		84		43		45		47		56		50

Table 9. Comparison of the average number of features on different improved transfer functions. Significant values are in bold.

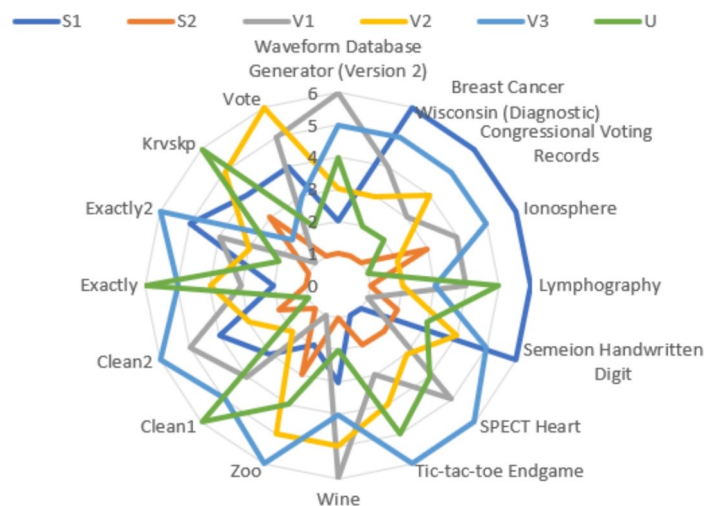


Fig. 8. Ranking distribution of the classification errors on different improved transfer functions.

features in 4 out of 16 datasets and securing a total rank score of 43. This highlights its efficiency in reducing feature dimensionality. The transfer function i_V_1 also performs well, with a total rank score of 45, and it excels in datasets like Waveform Database Generator (Version 2) and Breast Cancer Wisconsin (Diagnostic), where it achieves the lowest number of features. In contrast, the transfer function i_S_1 shows the poorest performance, with the highest average number of features in several datasets and a total rank score of 84. The transfer functions i_V_2 , i_V_3 , and i_U exhibit moderate performance, with total rank scores of 47, 56, and 50, respectively. Notably, i_V_2 performs well in the Semeion Handwritten Digit and SPECT Heart datasets, while i_U excels in the Ionosphere and Krvskp datasets. Overall, the results indicate that i_S_2 is the most effective transfer function for minimizing the number of features, while i_S_1 lags behind in performance. These findings provide valuable insights into the selection of transfer functions for optimizing feature selection tasks.

Effectiveness of improved transfer functions

Compared with the original transfer functions, some experiments of the six improved transfer functions on ten test datasets had been verified. Table 10 shows the comparison results of classification error between the original transfer functions and the improved transfer functions, $i_$ means the improved transfer functions. Table 11 compares the average number of features.

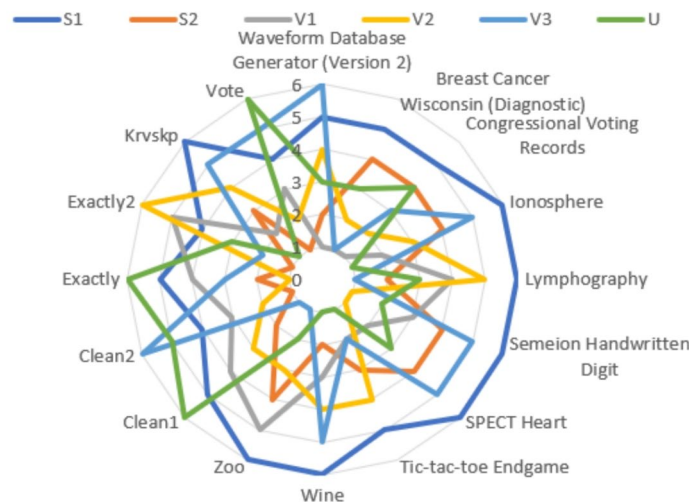


Fig. 9. Ranking distribution of the average number of features for different improved transfer functions.

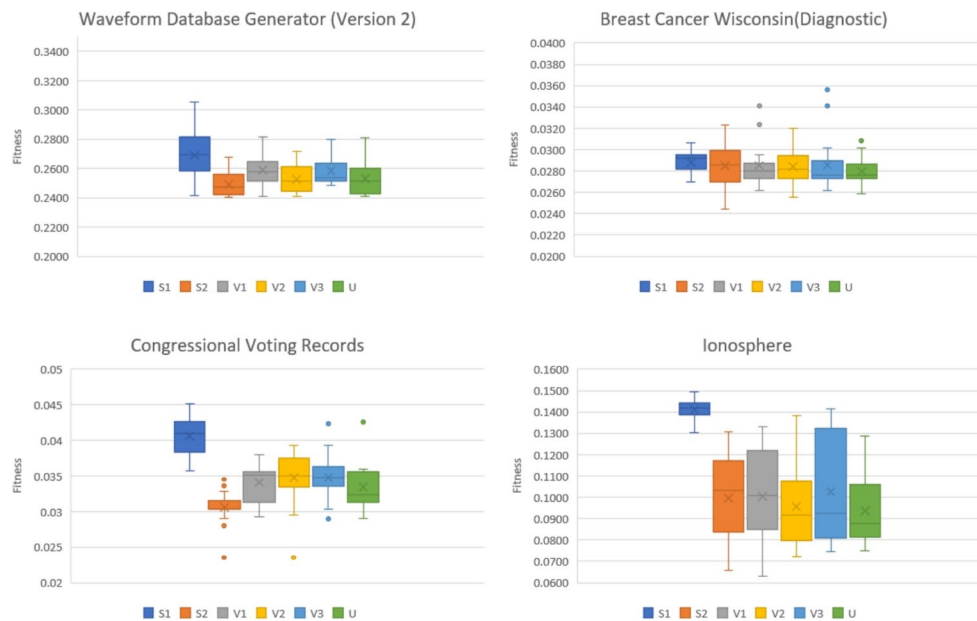


Fig. 10. The boxplots of different improved transfer functions (1).

From Table 10, the improved transfer functions generally outperform the original transfer functions in terms of classification error across most datasets. The improved transfer function i_S2 demonstrates the best overall performance, achieving the lowest classification errors in several datasets, such as Waveform Database Generator (Version 2), Breast Cancer Wisconsin (Diagnostic), and Exactly. This highlights its effectiveness in enhancing classification accuracy. The improved transfer function i_V1 also performs well, particularly in datasets like Ionosphere and Krvsdp, where it achieves the lowest classification errors. In contrast, the original transfer functions, such as $S1$ and $V3$, generally show higher classification errors, indicating their limitations in achieving optimal performance. The improved transfer functions i_V2 , i_V3 , and i_U exhibit moderate performance, with notable improvements over their original counterparts in datasets like Clean1 and Clean2. Overall, the results indicate that the improved transfer functions, particularly i_S2 and i_V1 , are more effective in reducing classification errors compared to the original transfer functions.

In Table 11, the improved transfer functions generally outperform the original transfer functions in terms of the average number of features selected across most datasets. The improved transfer function i_S2 demonstrates the best overall performance, achieving the lowest average number of features in several datasets, such as Waveform Database Generator (Version 2), Congressional Voting Records, and Clean1. This highlights its efficiency in reducing feature dimensionality. The improved transfer function i_V1 also performs well,

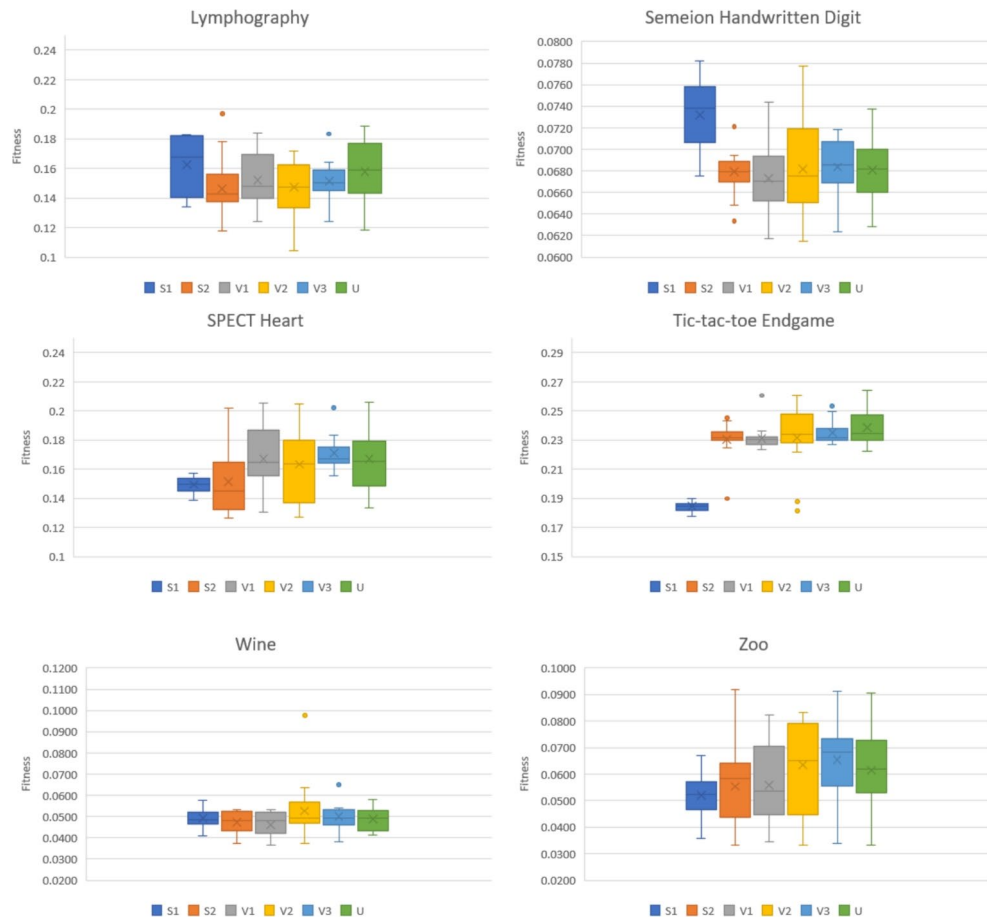


Fig. 11. The boxplots of different improved transfer functions (2).

particularly in datasets like Breast Cancer Wisconsin (Diagnostic) and Krvskp, where it achieves the lowest number of features. In contrast, the original transfer functions, such as S_1 and V_3 , generally show higher average numbers of features, indicating their limitations in achieving optimal performance. The improved transfer functions i_V_2 , i_V_3 , and i_U exhibit moderate performance, with notable improvements over their original counterparts in datasets like Semeion Handwritten Digit and Exactly2. Overall, the results indicate that the improved transfer functions, particularly i_S_2 and i_V_1 , are more effective in minimizing the number of features compared to the original transfer functions. These findings underscore the importance of refining transfer functions to optimize feature selection tasks and improve model efficiency.

Effectiveness of the nonlinear adaptive convergence factor

In this experiment, in order to test the effectiveness of the convergence factor (a), it is set to three fixed values of 0.5, 1, and 1.5, respectively. These three fixed values are compared with the nonlinear adaptive strategy proposed in this paper. We compare the errors and number of features obtained in the above four situations. The experimental results are shown in Table 12. Figure 13 shows the average classification error obtained by GWO-SRS with different parameter (a) values, and Fig. 14 shows the average number of features obtained by GWO-SRS with different parameter (a) values.

From Table 12, the self-adaptive convergence factor (a) demonstrates superior performance in both classification error and the number of features selected across most datasets. It achieves the lowest classification errors in datasets such as Waveform Database Generator (Version 2), Clean1, Clean2, Exactly, Exactly2, and Vote, while also selecting fewer features in many cases, such as in Congressional Voting Records, Ionosphere, and Krvskp. The fixed convergence factors (0.5, 1, and 1.5) show varying performance, with 1.5 occasionally performing well in reducing classification errors, as seen in SPECT Heart and Krvskp. However, the self-adaptive approach consistently outperforms the fixed values, highlighting its effectiveness in balancing exploration and exploitation during the optimization process. This adaptability makes the self-adaptive convergence factor a robust choice for improving both accuracy and efficiency in feature selection tasks.

Effectiveness of the learning strategies based on the head wolf plunder

In this section, we test wolf pack learning strategies based on the head wolf plunder. It compares the strategies we propose in this paper with traditional individual update strategy, roulette strategy, and adaptive update with the number of iterations strategy. Experimental results are shown in the Table 13. Among them, Traditional

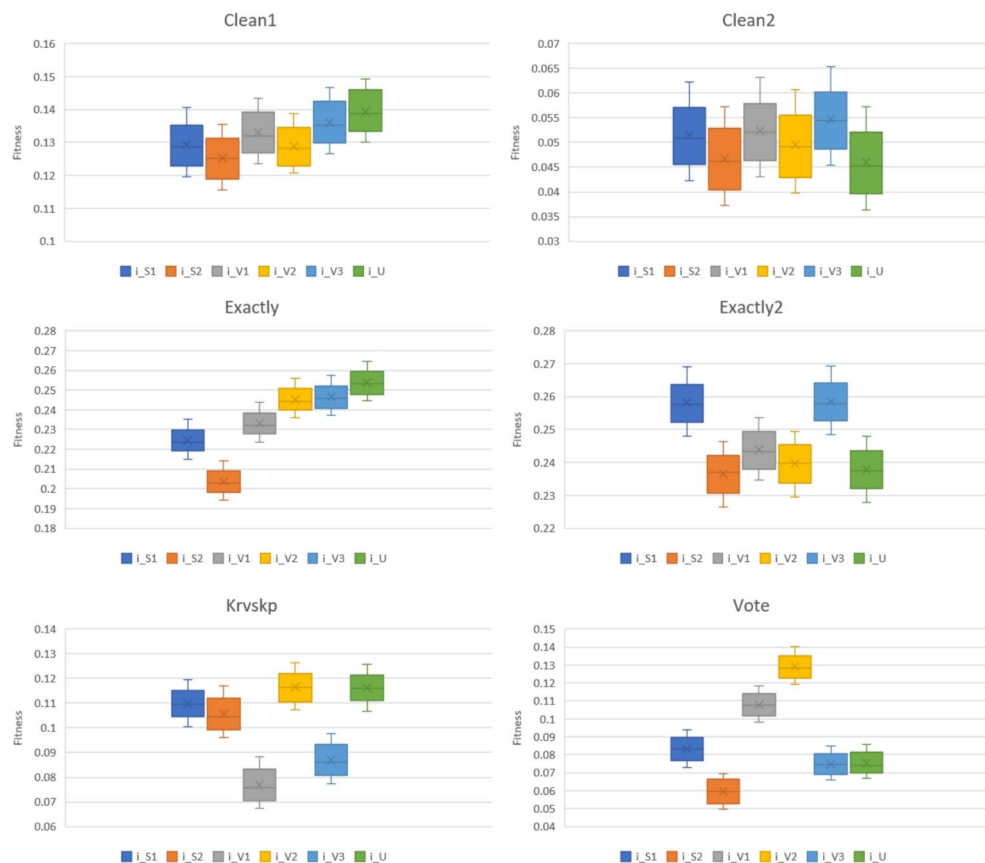


Fig. 12. The boxplots of different improved transfer functions (3).

Dataset	S_1	i_{S_1}	S_2	i_{S_2}	V_1	i_{V_1}	V_2	i_{V_2}	V_3	i_{V_3}	U	i_U
Waveform Database Generator (Version 2)	0.2811	0.2462	0.2731	0.2348	0.2800	0.2624	0.3471	0.2773	0.2971	0.2634	0.2879	0.2774
Breast Cancer Wisconsin (Diagnostic)	0.0292	0.0288	0.0288	0.0281	0.0289	0.0285	0.0290	0.0284	0.0292	0.0286	0.0297	0.0280
Congressional Voting Records	0.0412	0.0406	0.0311	0.0306	0.0353	0.0340	0.0341	0.0347	0.0333	0.0348	0.0335	0.0334
Ionosphere	0.1393	0.1410	0.1259	0.0998	0.1008	0.1005	0.1033	0.0956	0.1111	0.1026	0.1005	0.0938
Lymphography	0.1643	0.1624	0.1529	0.1460	0.1617	0.1523	0.1537	0.1474	0.1521	0.1513	0.1588	0.1579
Semeion Handwritten Digit	0.0721	0.0732	0.0693	0.0680	0.0676	0.0673	0.0692	0.0682	0.0692	0.0684	0.0696	0.0681
SPECT Heart	0.1569	0.1494	0.1686	0.1513	0.1609	0.1673	0.1600	0.1634	0.1642	0.1713	0.1639	0.1670
Tic-Tac-Toe Endgame	0.1857	0.1845	0.1979	0.2305	0.2336	0.2312	0.2387	0.2318	0.2552	0.2553	0.2399	0.2387
Wine	0.0494	0.0494	0.0481	0.0474	0.0523	0.0560	0.0532	0.0527	0.0564	0.0503	0.0500	0.0488
Zoo	0.0598	0.0519	0.0467	0.0554	0.0655	0.0462	0.0637	0.0635	0.0676	0.0654	0.0659	0.0615
Clean1	0.1684	0.1346	0.1593	0.1162	0.1876	0.1586	0.1752	0.1367	0.1462	0.1683	0.1991	0.1457
Clean2	0.0951	0.0480	0.1001	0.0476	0.0890	0.4721	0.0638	0.0579	0.1217	0.0504	0.0761	0.0372
Exactly	0.2647	0.2963	0.3584	0.2043	0.3971	0.2110	0.2908	0.2135	0.2730	0.2516	0.3687	0.2043
Exactly2	0.4162	0.2530	0.3705	0.2260	0.2417	0.2603	0.3194	0.2203	0.3051	0.2410	0.2814	0.2406
Krvskp	0.1064	0.1131	0.1564	0.0846	0.0905	0.0677	0.1776	0.1085	0.1951	0.0860	0.2063	0.1360
Vote	0.0942	0.0873	0.1064	0.0631	0.1957	0.1023	0.1917	0.1120	0.1052	0.0631	0.1130	0.0715

Table 10. Comparison of improved transfer functions and original transfer functions on the classification error. Significant values are in bold.

represents traditional random crossover; Adaptive represents adaptive with several iterations. Among them, Fig. 15 shows the average classification error obtained by different updating strategies, and Fig. 16 shows the average number of features obtained by different updating strategies.

As seen from Table 13, the plunder strategy demonstrates superior performance in both classification error and the number of features selected across most datasets. It achieves the lowest classification errors in datasets such as Waveform Database Generator (Version 2), Breast Cancer Wisconsin (Diagnostic), Lymphography,

Dataset	S_1	i_S1	S_2	i_S2	V_1	i_V1	V_2	i_V2	V_3	i_V3	U	i_U
Waveform Database Generator (Version 2)	26.55	25.15	28.15	28.40	30.40	29.90	31.35	30.25	32.50	31.00	34.60	28.10
Breast Cancer Wisconsin (Diagnostic)	13.60	13.95	12.30	12.70	11.85	11.35	12.10	12.40	11.70	11.35	11.85	12.50
Congressional Voting Records	7.75	7.30	4.75	3.85	3.35	3.10	3.55	3.25	3.45	3.35	3.95	3.85
Ionosphere	22.50	21.95	20.00	8.45	8.65	7.95	8.35	8.20	9.10	9.40	8.50	7.30
Lymphography	13.80	13.55	10.95	6.45	6.35	6.75	7.30	7.05	7.10	6.25	7.10	6.70
Semeion Handwritten Digit	153.75	151.80	151.40	128.75	124.80	128.15	128.70	127.02	126.75	130.60	127.95	127.75
SPECT Heart	18.90	18.60	13.80	9.75	9.80	8.65	9.45	8.30	9.95	9.85	9.45	9.40
Tic-Tac-Toe Endgame	9.00	9.00	8.00	5.20	5.00	4.90	5.95	5.40	5.00	4.90	5.35	4.75
Wine	12.35	9.65	7.20	4.20	4.40	4.45	4.25	4.55	4.50	4.60	5.00	4.10
Zoo	9.10	12.50	10.40	7.05	8.80	8.00	7.00	6.75	6.90	6.65	6.80	6.60
Clean1	101.62	98.75	96.51	90.03	98.70	94.57	97.16	91.02	94.86	90.63	110.00	95.66
Clean2	99.17	94.55	96.03	91.12	98.23	92.00	95.87	90.22	109.75	98.54	101.51	96.25
Exactly	10.42	9.11	8.56	8.91	10.70	8.06	9.10	6.54	9.15	7.25	12.67	10.51
Exactly2	9.26	7.95	8.79	5.30	10.19	8.53	9.56	9.73	9.52	6.17	8.00	6.29
Krvskp	23.40	19.53	21.42	19.16	20.80	18.65	25.63	17.05	20.66	21.02	20.76	16.54
Vote	10.57	7.55	9.76	6.15	8.97	5.22	11.57	9.01	10.56	7.99	9.68	8.14

Table 11. Comparison of improved transfer functions and original transfer functions on the average number of features. Significant values are in bold.

Convergence factor(a)	0.5		1		1.5		self-adaption	
	Error	Number	Error	Number	Error	Number	Error	Number
Waveform Database Generator (Version 2)	0.2651	31.20	0.2861	30.35	0.2557	31.30	0.2333	28.65
Breast Cancer Wisconsin (Diagnostic)	0.0274	11.00	0.0276	12.60	0.0275	11.05	0.0271	11.05
Congressional Voting Records	0.0314	3.90	0.0317	4.30	0.0320	4.40	0.0309	3.80
Ionosphere	0.0991	9.90	0.1189	14.35	0.1116	11.15	0.1018	9.00
Lymphography	0.1451	6.60	0.1391	8.30	0.1514	8.80	0.1434	6.60
Semeion Handwritten Digit	0.0676	133.10	0.0688	139.35	0.0692	144.85	0.0675	129.40
SPECT Heart	0.1563	9.50	0.1518	10.15	0.1459	10.50	0.1553	9.35
Tic-Tac-Toe Endgame	0.2336	5.95	0.2295	5.50	0.2297	5.50	0.2291	5.20
Wine	0.0466	4.20	0.0458	5.35	0.0449	4.85	0.0460	5.40
Zoo	0.0520	7.10	0.0539	7.85	0.0482	8.90	0.0516	7.25
Clean1	0.1397	90.57	0.1462	91.11	0.1498	89.61	0.1143	87.45
Clean2	0.0764	92.91	0.0597	92.67	0.0601	90.53	0.0481	91.05
Exactly	0.2263	9.01	0.2346	8.97	0.2197	9.53	0.2014	8.41
Exactly2	0.2863	5.87	0.2894	7.21	0.2576	6.25	0.2541	5.91
Krvskp	0.1195	21.59	0.1081	20.18	0.0922	19.64	0.0931	17.91
Vote	0.0957	8.94	0.0891	7.80	0.0759	6.79	0.0615	6.21

Table 12. Regarding the testing of convergence factor(a). Significant values are in bold.

Clean1, Exactly, Exactly2, and Krvskp, while also selecting fewer features in many cases, such as in Waveform Database Generator (Version 2), Lymphography, and Clean1. The adaptive and roulette strategies show varying performance, with the roulette strategy occasionally performing well in reducing classification errors, as seen in Ionosphere, Clean2, and Vote. However, the plunder strategy consistently outperforms the traditional, adaptive, and roulette strategies, highlighting its effectiveness in balancing exploration and exploitation during the optimization process. This makes the plunder strategy a robust choice for improving both accuracy and efficiency in feature selection tasks.

Conclusion

This article introduces a new feature selection algorithm (GWO-SRS) for feature selection. GWO-SRS improves the wolf pack hierarchy and proposes a learning strategy of self-repulsion for elite leader wolves to reduce the number of selected features. It proposes an adaptive strategy with nonlinear convergence factors and improves the transfer functions to improve the convergence speed of the algorithm and avoid falling into local optima. Meanwhile, a wolf pack learning strategy based on head wolf plunder is proposed to enhance the weight of learning from head wolf.

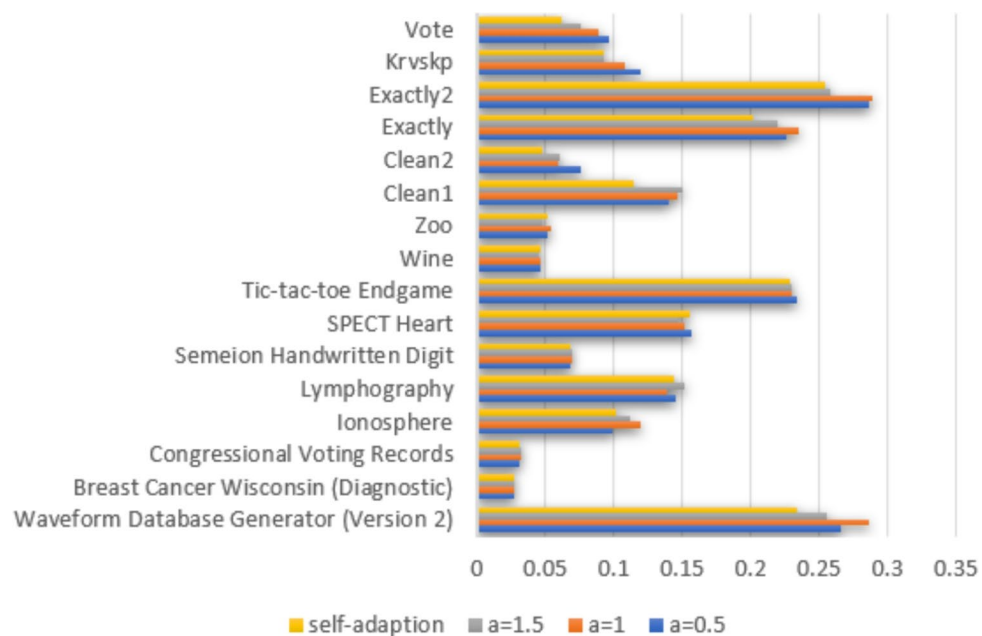


Fig. 13. The average classification error on different parameter (a) values.

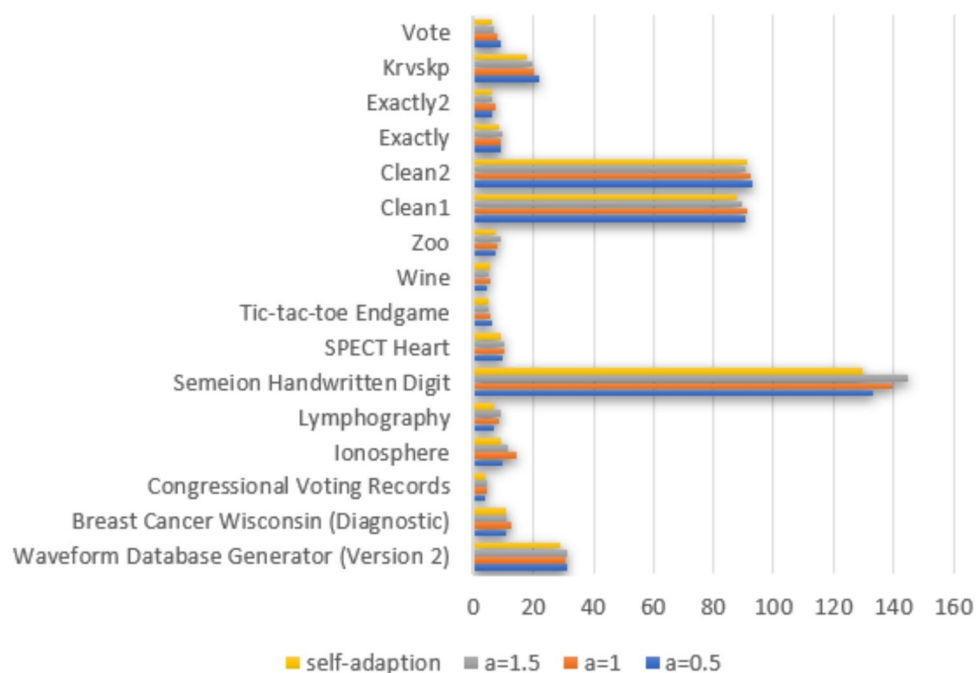


Fig. 14. The average number of features obtained on different parameter (a) values.

The experiment used the UCI test datasets to verify the performance of GWO-SRS. The simulation results show that the proposed algorithm has better classification accuracy than the other five relevant algorithms, and its performance is particularly outstanding in terms of the average number of features.

In future work, we can focus on optimizing the complexity of the algorithm and developing scalable feature selection algorithms capable of handling ultra-high-dimensional datasets, potentially through the integration of dimensionality reduction techniques or parallel computing frameworks. Additionally, we aim to explore adaptive parameter tuning mechanisms to enhance the algorithm's robustness and efficiency across diverse datasets. Another key direction will be to evaluate and improve the performance of GWO-SRS on noisy and imbalanced datasets, which are common in real-world applications. Furthermore, we plan to extend the application of feature selection methods to other fields, such as bioinformatics, image processing, and industrial fault detection, to

Update strategy	Traditional		Adaptive		Roulette		plunder strategy	
	Error	Number	Error	Number	Error	Number	Error	Number
Waveform Database Generator (Version 2)	0.2551	29.65	0.2751	31.45	0.2459	30.41	0.2449	27.30
Breast Cancer Wisconsin (Diagnostic)	0.0275	12.65	0.0277	13.00	0.0279	12.75	0.0270	12.80
Congressional Voting Records	0.0307	4.20	0.0310	3.85	0.0304	4.05	0.0307	3.90
Ionosphere	0.1035	9.00	0.0971	8.05	0.0891	7.05	0.1012	9.15
Lymphography	0.1490	7.40	0.1502	7.10	0.1506	7.35	0.1453	7.05
Semeion Handwritten Digit	0.0678	129.20	0.0674	127.80	0.0677	129.20	0.0682	127.55
SPECT Heart	0.1663	9.80	0.1537	9.70	0.1509	9.75	0.1649	9.20
Tic-Tac-Toe Endgame	0.2271	5.20	0.2333	4.85	0.2327	7.95	0.2258	5.40
Wine	0.0470	4.80	0.0447	4.65	0.0471	4.55	0.0470	4.50
Zoo	0.0535	6.65	0.0538	6.80	0.0589	6.45	0.0531	7.20
Clean1	0.1368	90.55	0.1341	88.16	0.1499	88.94	0.1276	87.35
Clean2	0.0675	91.45	0.0881	90.70	0.0516	90.15	0.0522	89.57
Exactly	0.2396	10.08	0.2234	9.41	0.2153	8.76	0.2045	8.93
Exactly2	0.2947	8.60	0.2855	5.81	0.2640	6.79	0.2548	5.82
Krvskp	0.1350	20.05	0.1086	18.51	0.1097	19.70	0.0912	18.56
Vote	0.0961	8.52	0.0864	7.18	0.0698	6.94	0.0710	6.53

Table 13. Test on the learning strategy of head wolf plunder. Significant values are in bold.

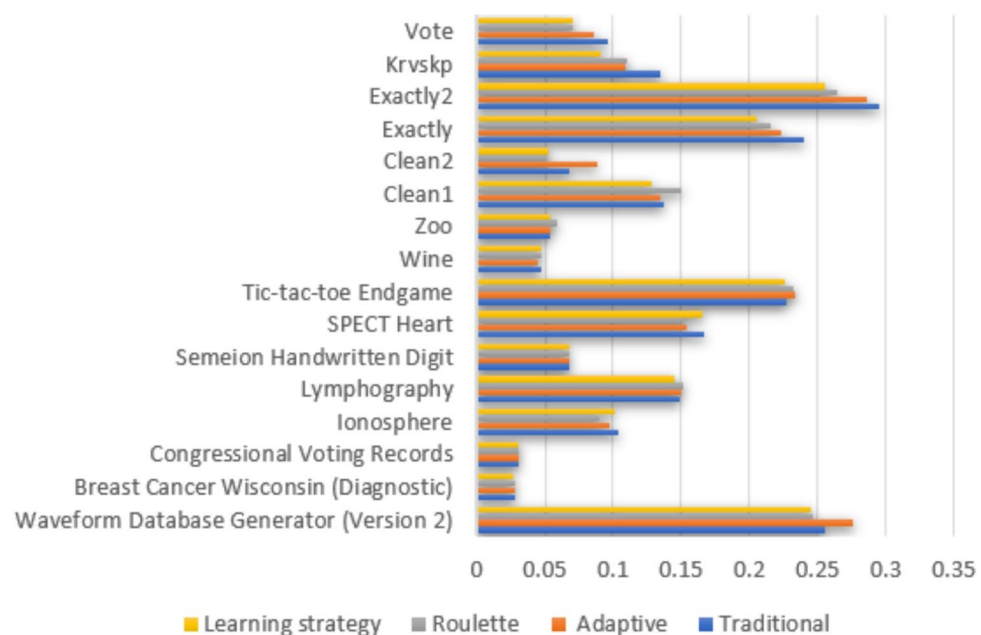


Fig. 15. The average classification error obtained by different updating strategies.

demonstrate their versatility and effectiveness in solving complex problems across various domains. These efforts will not only address the current limitations of GWO-SRS but also broaden its applicability and impact in both academic and practical settings.

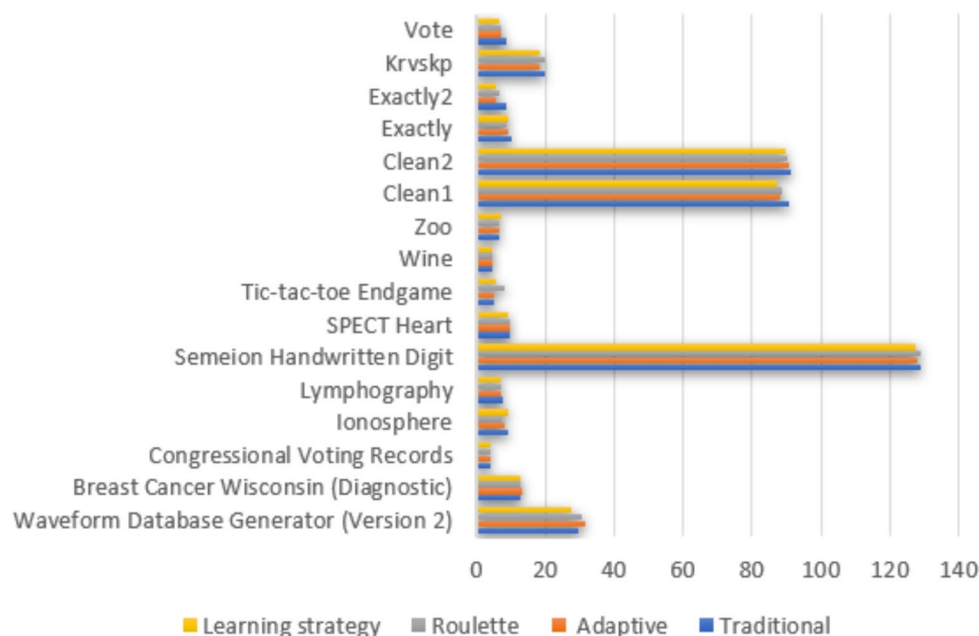


Fig. 16. The average number of features obtained by different updating strategies.

Data availability

The data used to support the findings of this study are available from the corresponding author upon request.

Received: 15 November 2024; Accepted: 3 April 2025

Published online: 14 April 2025

References

1. Zebari, R., Abdulazeez, A., Zeebaree, D., Zebari, D. & Saeed, J. A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction. *J. Appl. Sci. Technol. Trends* **1**(1), 56–70 (2020).
2. Abasi, A. K., Khader, A. T., Al-Betar, M. A., Nairn, S., Makhadmeh, S. N. & Alyasseri, Z. A. A. An improved text feature selection for clustering using binary grey wolf optimizer. In *National Technical Seminar on Unmanned System Technology* (2019).
3. Vharkate, M. N. & Musande, V. B. Fusion based feature extraction and optimal feature selection in remote sensing image retrieval. *Multimed. Tools Appl.* **81**(22), 31787–31814 (2022).
4. Cheng, Z.-H., Shang, H. & Qian, C. Detection-rate-emphasized multi-objective evolutionary feature selection for network intrusion detection. *arXiv preprint arXiv:2406.09180* (2024).
5. Gupta, P., Alok, A. K. & Sharma, V. Advancing gene expression data analysis: An innovative multi-objective optimization algorithm for simultaneous feature selection and clustering. *Braz. Arch. Biol. Technol.* **67**, 24230508 (2024).
6. Thabtah, F., Kamalov, F., Hammoud, S. & Shahamiri, S. R. Least loss: A simplified filter method for feature selection. *Inf. Sci.* **534**, 1–15 (2020).
7. Alzaqebah, M., Alrefai, N., Ahmed, E. A., Jawarneh, S. & Alsmadi, M. K. Neighborhood search methods with moth optimization algorithm as a wrapper method for feature selection problems. *Int. J. Electr. Comput. Eng.* **10**(4), 2088–8708 (2020).
8. Liu, H., Zhou, M. & Liu, Q. An embedded feature selection method for imbalanced data classification. *IEEE/CAA J. Autom. Sin.* **6**(3), 703–715 (2019).
9. Bommert, A., Sun, X., Bischl, B., Rahnenführer, J. & Lang, M. Benchmark for filter methods for feature selection in high-dimensional classification data. *Comput. Stat. Data Anal.* **143**, 106839 (2020).
10. González, J., Ortega, J., Damas, M., Martín-Smith, P. & Gan, J. Q. A new multi-objective wrapper method for feature selection-accuracy and stability analysis for bci. *Neurocomputing* **333**, 407–418 (2019).
11. Zhang, D. et al. [retracted] heart disease prediction based on the embedded feature selection method and deep neural network. *J. Healthc. Eng.* **2021**(1), 6260022 (2021).
12. Agrawal, P., Abutarboush, H. F., Ganesh, T. & Mohamed, A. W. Metaheuristic algorithms on feature selection: A survey of one decade of research (2009–2019). *IEEE Access* **9**, 26766–26791 (2021).
13. Rostami, M., Berahmand, K., Nasiri, E. & Forouzandeh, S. Review of swarm intelligence-based feature selection methods. *Eng. Appl. Artif. Intell.* **100**, 104210 (2021).
14. Too, J. & Abdullah, A. R. A new and fast rival genetic algorithm for feature selection. *J. Supercomput.* **77**(3), 2844–2874 (2021).
15. Rostami, M., Berahmand, K. & Forouzandeh, S. A novel community detection based genetic algorithm for feature selection. *J. Big Data* **8**(1), 2 (2021).
16. Barhoush, M., Abed-Alguni, B. H. & Al-Qudah, N. Improved discrete salp swarm algorithm using exploration and exploitation techniques for feature selection in intrusion detection systems. *J. Supercomput.* **79**(18), 21265–21309 (2023).
17. Faris, H., Mafarja, M. M., Heidari, A. A., Aljarah, I. & Fujita, H. An efficient binary salp swarm algorithm with crossover scheme for feature selection problems. *Knowl. Based Syst.* **154** 43–67. <https://doi.org/10.1016/j.knsys.2018.05.009> (2018).
18. Wang, G. Q., Jia, J. B. & Li, X. Y. Research on feature selection based on improved particle swarm optimization. *Adv. Mater. Res.* **591–593**, 2651–2654 (2012).
19. Aghdam, M. H., Ghasem-Aghaee, N. & Basiri, M. E. Text feature selection using ant colony optimization. *Expert Syst. Appl.* **36**(3p2), 6843–6853 (2009).
20. Allvi, M. W., Hasan, M., Rayan, L., Shahabuddin, M. & Ibrahim, M. Feature selection for learning-to-rank using simulated annealing. *Int. J. Adv. Comput. Sci. Appl.* **11**(3), 699–705 (2020).

21. Hou, Y., Gao, H., Wang, Z. & Du, C. Improved grey wolf optimization algorithm and application. *Sensors* **22**(10), 3810 (2022).
22. Mirjalili, S. M., Mirjalili, S. M. & Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014).
23. Abdel-Basset, M. et al. An improved binary grey-wolf optimizer with simulated annealing for feature selection. *IEEE Access* **9**, 139792–139822. <https://doi.org/10.1109/ACCESS.2021.3117853> (2021).
24. Al-Wajih, R., Abdulkadir, S. J., Aziz, N., Al-Tashi, Q. & Talpur, N. Hybrid binary grey wolf with harris hawks optimizer for feature selection. *IEEE Access* **9**, 31662–31677 (2021). <https://doi.org/10.1109/ACCESS.2021.3060096>
25. Al-Tashi, Q., Abdul Kadir, S. J., Rais, H. M., Mirjalili, S. & Alhussien, H. Binary optimization using hybrid grey wolf optimization for feature selection. *IEEE Access* **7**, 39496–39508. <https://doi.org/10.1109/ACCESS.2019.2906757> (2019).
26. Kazem, M., Amirpouya, H., Seyedali, M. & Amir, B. F. Adaptive grey wolf optimizer. *Neural Comput. Appl.* **34**(10), 7711–7731 (2022).
27. Abdel-Basset, M., El-Shahat, D., El-Henawy, I., De Albuquerque, V. H. C. & Mirjalili, S. A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection. *Expert Syst. Appl.* **139**, 112824–11282414 (2020).
28. Al-Wajih, R., Abdulkadir, S. J., Aziz, N. S. B. A. & Al-Tashi, Q. Binary grey wolf optimizer with k-nearest neighbor classifier for feature selection. In *2020 International Conference on Computational Intelligence (ICCI)* (2020).
29. Too, J. & Abdullah, A. R. Opposition based competitive grey wolf optimizer for EMG feature selection. *Evol. Intell.* **14**(4), 1691–1705 (2020).
30. Latha, R. S., Sreekanth, G. R., Suganth, R. C. & Geetha, M. Hybrid binary gray wolf optimization for finding optimal features in classification problems (2019).
31. Narinder, S. & Singh, S. B. Hybrid algorithm of particle swarm optimization and grey wolf optimizer for improving convergence performance. *J. Appl. Math.* **2017**, 1–15 (2017).
32. Abasi, A. K., Khader, A. T., Al-Betar, M. A., Naim, S. & Alyasseri, Z. A. A. An improved text feature selection for clustering using binary grey wolf optimizer (2021).
33. Hu, P., Pan, J.-S. & Chu, S. C. Improved binary grey wolf optimizer and its application for feature selection. *Knowl. Based Syst.* **195**, 105746 (2020).
34. Wang, D., Ji, Y., Wang, H. & Huang, M. Binary grey wolf optimizer with a novel population adaptation strategy for feature selection. *IET Control Theory Appl.* **17**(17), 2313–2331 (2023).
35. Tripathi, A., Bharti, K. K. & Ghosh, M. A fusion of binary grey wolf optimization algorithm with opposition and weighted positioning for feature selection. *Int. J. Inf. Technol.* **8**, 15 (2023).
36. Yang, D., Zhou, C., Wei, X., Chen, Z. & Zhang, Z. Multi-strategy assisted multi-objective whale optimization algorithm for feature selection. *Comput. Model. Eng. Sci.* **140**(8), 1563–1593 (2024).
37. Hussien, A. G., Hassanien, A. E., Houssein, E. H., Bhattacharyya, S. & Amin, M. S-shaped binary whale optimization algorithm for feature selection (2019).
38. Azar, A. T., Banu, N. & Koubaa, A. Rough set based ant-lion optimizer for feature selection. In *International Conference on Devices, Circuits and Systems* (2020).
39. Vashishtha, G. & Kumar, R. Feature selection based on gaussian ant lion optimizer for fault identification in centrifugal pump (2023).
40. Sun, L., Qin, H., Przystupa, K., Cui, Y., Kochan, O., Skowron, M. & Su, J. A hybrid feature selection framework using improved sine cosine algorithm with metaheuristic techniques. *Energies* **15** 3485. <https://www.mdpi.com/1996-1073/15/10/3485> (2022).
41. Kale, G. A. & Yüzge, U. Advanced strategies on update mechanism of sine cosine optimization algorithm for feature selection in classification problems. *Eng. Appl. Artif. Intell.* **107**, 104506 (2022).
42. Li, M., Liu, Y., Zheng, Q., Qin, W. & Ren, X. Stable feature selection based on brain storm optimisation for high-dimensional data. *Electron. Lett.* **58**(1), 10–12 (2022).
43. Xue, Y. & Zhao, Y. Structure and weights search for classification with feature selection based on brain storm optimization algorithm. *Appl. Intell.* **52**(5), 5857–5866 (2022).
44. Emary, E., Zawbaa, H. M. & Hassanien, A. E. Binary grey wolf optimization approaches for feature selection. *Neurocomputing* **172**, 371–381 (2016).
45. Arora, S. & Anand, P. Binary butterfly optimization approaches for feature selection. *Expert Syst. Appl.* **116**, 147–160. <https://doi.org/10.1016/j.eswa.2018.08.051> (2019).

Acknowledgements

This work was supported in part by the Research and Practice Project of Research Teaching Reform in Henan Undergraduate University under Grant 2022SYJXLX114, in part by the Key Research Programs of Higher Education Institutions in Henan Province under Grant 24B520026, in part by the Special Research Project for the Construction of Provincial Demonstration Schools at Nanyang Institute of Technology under Grant SFX2-02314, and in part by the Interdisciplinary Sciences Project, Nanyang Institute of Technology.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.Y.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025